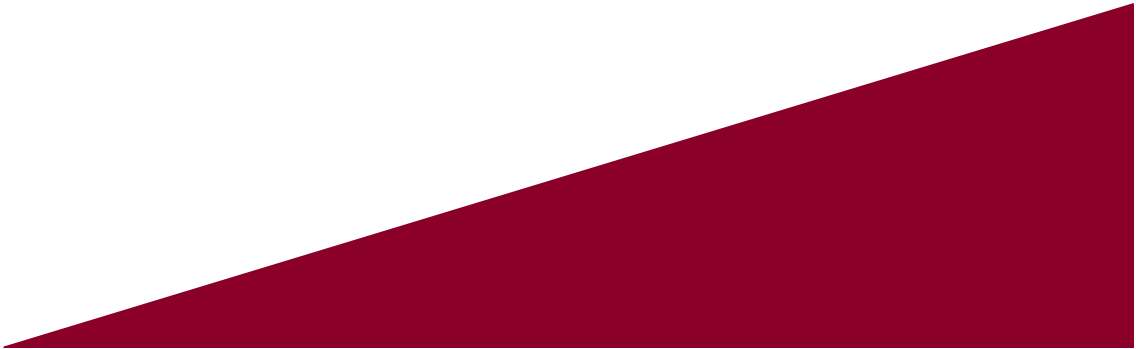




인공지능

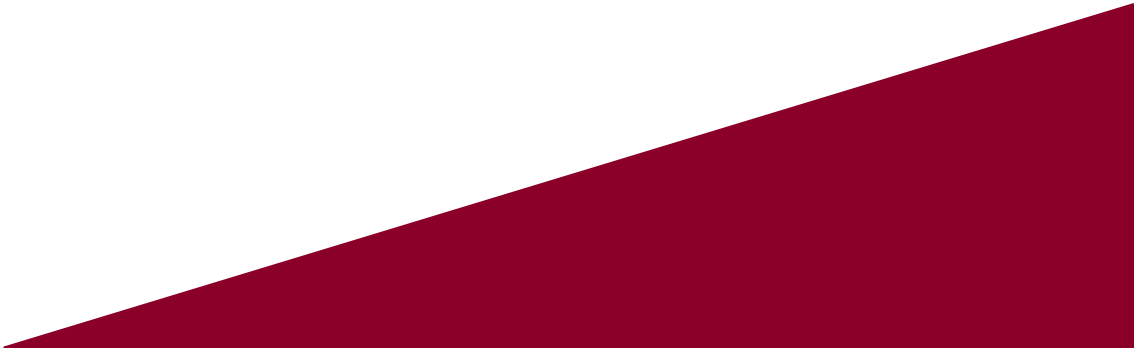
Artificial Intelligence





시계열 데이터와 순환 신경망

본 강의자료는 한빛아카데미 <파이썬으로 만드는 인공지능>을 내용을 기반으로 제작 및 보완되었음을 알립니다.

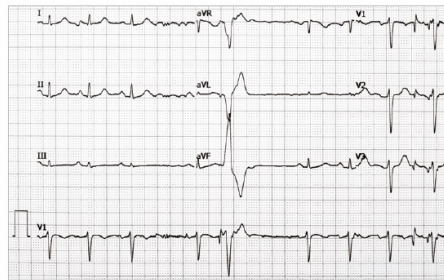


8.1 시계열 데이터의 이해

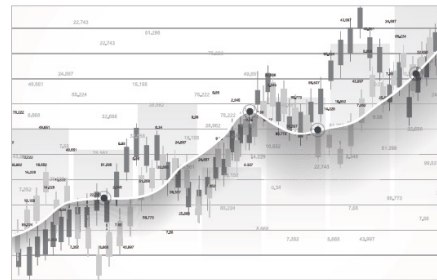
- 시계열 데이터는 시간 축을 따라 신호가 변하는 동적 데이터임
- 지금까지 배운 SVM, MLP, DNN, CNN은 정적 데이터를 한꺼번에 입력 받으므로 시계열 데이터를 처리하는데 적합하지 않음
- 시계열 데이터를 고정 길이의 정적 데이터로 바꾸는 기법을 적용할 수도 있지만 정보 손실이 크고 변환 기법을 사람이 설계해야 하는 부담이 있음
 - 영상 정보를 1D로 가공하여 DNN 입력으로 사용할 수 있으나 정보 손실이 큰 이유와 동일
- 딥러닝에서는 시계열 특성을 십분 활용하는 순환 신경망(RNN, Recurrent Neural Network) 또는 장단기메모리(LSTM, Long Short Term Memory)을 사용함

8.1 시계열 데이터의 이해

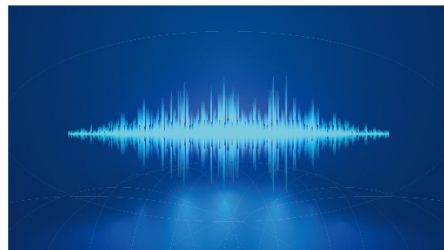
- [그림 8-1]은 다양한 시계열 데이터의 예
 - (a) 심전도 신호를 분석하는 의료 인공지능을 만들어 더욱 정확하게 병을 진단할 수 있음
 - (b) 주식 시세를 잘 분석하는 인공지능 제품을 만들면 투자에 유리할 수 있음
 - (c) 정확률이 높은 음성 인식기를 만들면 경쟁력 있는 인공지능 제품을 만들 수 있음
 - (d) 유전자 분석하는 인공지능을 만들면 신약 개발에 활용할 수 있음



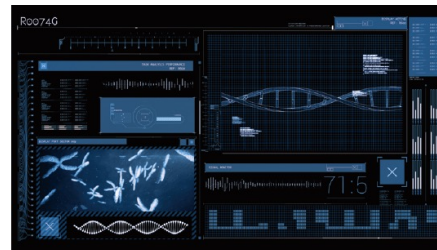
(a) 심전도



(b) 주식 시세



(c) 음성 인식 데이터



(d) 유전자 염기 서열

그림 8-1 다양한 시계열 데이터

8.1.1 시계열 데이터의 특성

- 시계열 데이터의 특성 * 각 특성에 대해서 다음 장에 있음
 - 요소의 순서가 중요
 - 샘플의 길이가 다름
 - 문맥 의존성을 지님
 - 계절성을 가지는 데이터가 많음

8.1.1 시계열 데이터의 특성

- 요소의 순서가 중요

- “세상에는 시계열 데이터가 참 많다” 라는 문장을 “데이터가 시계열 많다 참 세상에는” 라는 문장으로 바꾸면 의미가 크게 훼손됨
- 반면, 시간 정보가 없는 데이터는 특징의 순서를 바꿔 표현해도 아무런 문제가 없음
- 예를 들어, iris 데이터에서는 샘플이 <꽃잎 길이, 꽃잎 너비, 꽃받침 길이, 꽃받침 너비>라는 특징 벡터로 표현되는데, 벡터의 요소 즉 특징의 순서를 바꿔 <꽃잎 너비, 꽃받침 너비, 꽃잎 길이, 꽃받침 길이>로 표현해도 상관없음

- 샘플의 길이가 다름

- 인공지능이란 단어를 짧게 발음하는 사람도 있고 인~공~지~능으로 길게 발음하는 사람도 있음
- 심전도를 켤 때도 심각한 환자는 더욱 정밀하게 진단하기 위해 더 길게 측정하기도 함
- 유전자 염기 서열에서도 잡음이 섞이거나 중간에 빠지는 염기가 발생해 측정할 때마다 길이가 조금씩 다를 수 있음

8.1.1 시계열 데이터의 특성

- 문맥 의존성을 지님
 - “시계열은 앞에서 말한 바와 같이 데이터를 구성하는 요소의 순서, 즉 나타나는 순서가 중요하다는 특성이 있다” 라는 문장에서 주어인 “시계열은”과 서술어인 “특성이 있다”는 밀접한 연관성이 있으며, 이런 연관성을 문맥 의존성 (context dependency)라고 함
 - 시계열을 처리하는 기계 학습 모델은 문맥 의존성을 스스로 발견하고 표현하는 능력이 있어야 함
 - 예시 문장에서 주어와 서술어 사이에 단어 12개가 있으므로 주어와 서술어는 12만큼 떨어져 있으며, 이처럼 연관성이 깊은 요소가 멀리 떨어져서 나타나는 상황을 장기 문맥 의존성 (long-term context dependency)라고 함
- 계절성을 가지는 데이터가 많음, seasonality
 - 상추 판매량은 야외 바비큐에 적절한 계절에 매출이 치솟는 경향이 있음
 - 수제 맥주 전문점은 금요일에 매출이 치솟는 경향이 있음
 - 미세먼지 수치나 항공권 판매량은 계절에 영향이 있음

8.1.1 시계열 데이터의 특성

- 시계열 데이터(동적데이터)의 정의

$$\mathbf{x} = (\mathbf{a}^1 \ \mathbf{a}^2 \cdots \mathbf{a}^t) \quad (8.1)$$

- 데이터의 길이가 가변적이므로 d 대신 t 로 표기함
- 벡터의 요가 벡터이므로 이를 고려하여 굵은 글씨체로 표기함 $\mathbf{a}^i$

- 시계열 데이터의 예

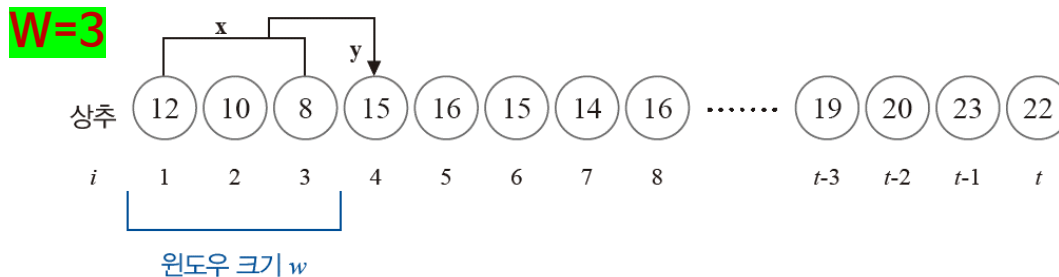
- 매일 기온, 습도, 미세먼지 농도를 기록한다면,
첫째날은 $\mathbf{a}^1 = (23.5, 42, 0.1)$, 둘째날은 $\mathbf{a}^2 = (25.5, 45, 0.08)$ 로 표현 가능
- 심전도의 경우 10개의 채널 값이 들어오고, 30초 분량의 심전도를 초당 20번 샘플링한다면
 $t=600$ 인 샘플 $\mathbf{x}$ 가 생성되는데, $\mathbf{x}$ 의 요소 $\mathbf{a}^i$ 는 10차원 벡터에 해당됨

8.1.2 미래 예측을 위한 데이터 준비

- 순환 신경망은 주로 미래 예측에 사용됨 (즉“추정” 문제 해결)
 - 내일의 주가 예측
 - 내일의 날씨 예측
 - 내일의 판매량 예측
- 분류와 추정을 종종 예측이라 부르기 때문에, 이를 구분할 수 있어야 함
 - 새로운 숫자 패턴이 입력되면 어떤 숫자인지 “분류 classification”하는 일도 “예측 prediction”
 - 판매 시계열 데이터를 보고 미래의 판매량을 “추정 forecasting”하는 일도 “예측 prediction”

8.1.2 미래 예측을 위한 데이터 준비

- 시계열 패턴 하나로 미래를 예측하는 신경망을 어떻게 학습할 것인가?
 - 미래를 예측하는 문제에서는 아래와 같이 일정한 길이로 패턴을 잘라 여러 샘플을 만들
 - 이때 이전 요소 몇 개를 학습에 사용할 것인가 의미하는 **윈도우 크기 w** 설정이 필요함



(a) 입력 데이터 샘플링

샘플	x	y
1	(12, 10, 8)	15
2	(10, 8, 15)	16
3	(8, 15, 16)	15
4	(15, 16, 15)	14
...	...	...
$t-w$	(19, 20, 23)	22

(b) 입력 패턴에서 생성한 샘플

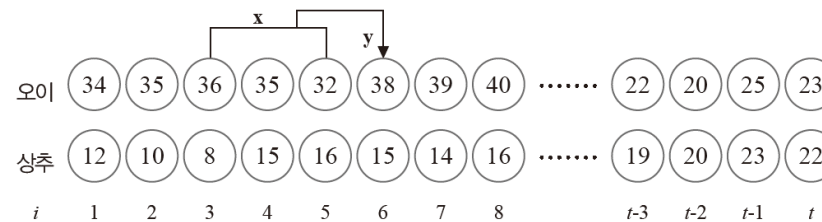
- 위 그림은 <이전 3일의 판매량을 보고 다음 날 판매량을 예측>하기 위한 **학습데이터 가공 예시**
 - $t=(1, 2, 3)$ 의 값을 보고 $t=4$ 순간 값을 예측하고,
 $t=(2, 3, 4)$ 의 값을 보고 $t=5$ 순간 값을 예측하는 방식으로 학습 데이터를 준비함

8.1.2 미래 예측을 위한 데이터 준비

- **시계열 패턴 하나로** 미래를 예측하는 신경망을 어떻게 학습할 것인가?
 - 〈이전 3일의 판매량을 보고 **다음 날 판매량을 예측**〉하기 위한 학습데이터 가공
 - $t=(1, 2, 3)$ 의 값을 보고 $t=4$ 순간 값을 예측하고,
 $t=(2, 3, 4)$ 의 값을 보고 $t=5$ 순간 값을 예측하는 방식으로 학습 데이터를 준비함
 - 〈이전 3일의 판매량을 보고 **이틀 후 판매량을 예측**〉하기 위한 학습데이터 가공
 - $t=(1, 2, 3)$ 의 값을 보고 $t=5$ 순간 값을 예측하고,
 $t=(2, 3, 4)$ 의 값을 보고 $t=6$ 순간 값을 예측하는 방식으로 학습 데이터를 준비함
- 얼마나 먼 미래를 예측할 것인지 지정하는 요인을 **수평선 계수 horizon factor**라고 하며, 위의 예시는 각각 수평선 계수 $h=1$ 혹은 $h=2$ 의 문제임
- 일반적으로, 수평선 계수가 클 수록 어려운 문제에 해당됨

8.1.2 미래 예측을 위한 데이터 준비

- 다중 시계열 패턴은 미래를 예측하는 신경망을 어떻게 학습할 것인가?
 - 품목이 2개 이상인 시계열 데이터를 동시에 처리해야 하는 다중 채널인 경우도 있음
 - 아래 그림은 다중 품목에 대한 <이전 3일의 판매량을 보고 다음 날 판매량을 예측>하기 위한 학습데이터 가공 예시임



(a) 입력 패턴

샘플	x	y
1	$((34, 12), (35, 10), (36, 8))$	$(35, 15)$
2	$((35, 10), (36, 8), (35, 15))$	$(32, 16)$
3	$((36, 8), (35, 15), (32, 16))$	$(38, 15)$
4	$((35, 15), (32, 16), (38, 15))$	$(39, 14)$
...	...	...
$t - w$	$((22, 19), (20, 20), (25, 23))$	$(23, 22)$

(b) 입력 패턴에서 생성한 샘플

그림 8-3 미래 예측 문제에서 샘플 생성하기(다중 채널)

8.1.3 시계열 데이터 사례: 비트코인 가격

- 비트코인 가격 데이터 다뤄보기
 - 디지털 통화에 관한 뉴스 사이트 코인데스크에서 비트코인 데이터를 다운로드
 - 얻는 방법 다시 해보니 약간 다른 듯?



(a) 코인데스크에서 데이터를 다운로드하기

	B	C	D	E	F	G
Date	Closing Pr	24h Open	24h High	24h Low (USD)		
2019-02-28	3772.94	3796.64	3824.17	3666.52		
2019-03-01	3799.68	3773.44	3879.23	3753.8		
2019-03-02	3811.61	3799.37	3840.04	3788.92		
2019-03-03	3804.42	3806.69	3819.19	3759.41		
2019-03-04	3782.66	3807.85	3818.7	3766.24		
2019-03-05	3689.86	3783.36	3804.35	3663.48		
2019-03-06	3832.08	3701.05	3866.72	3688.7		
2019-03-07	3848.96	3832.59	3881.97	3802.52		
2019-03-08	3859.84	3848.96	3890.75	3827.67		
2019-03-09	3828.37	3859.8	3918	3778.52		
2019-03-10	3898.87	3841.89	3948.88	3832.14		
2019-03-11	3899.66	3916.76	3921.93	3865.92		
2019-03-12	3851.25	3899.46	3913.46	3819.93		

(b) 비트코인 가격이 저장된 데이터 프레임

그림 8-4 코인데스크 사이트에서 다운로드한 비트코인 가격 데이터

*다운로드 사이트의 화면이나 메뉴가 바뀌는 경우는 빈번하여, 책과 다른 화면이 나올 수 있음

8.1.3 시계열 데이터 사례: 비트코인 가격

- 비트코인 가격 데이터 다뤄보기
 - (교재에서 제공하는) 파일을 열어보면, 아래의 그림과 같이 5개의 열이 있음
 - [Date, Closing Price, Open Price, High Price, Low Price] 로 구성 있음

```
[8]:
```

	closing price	open price	high price	low price
date				
2019- 02- 28	3816.6	3814.6	3883.7	3783.3
2019- 03- 01	3821.9	3816.7	3855.8	3816.4
2019- 03- 02	3823.1	3821.9	3843.2	3783.6
2019- 03- 03	3809.5	3823.2	3836.6	3789.7
2019- 03- 04	3715.9	3809.7	3828.4	3681.8

- 시계열 데이터 역시 csv로 담겨 있으므로, pandas 라이브러리로 데이터를 읽고, matplotlib 라이브러리로 시각화를 진행해 보겠음

8.1.3 시계열 데이터 사례: 비트코인 가격

■ 프로그램 8-1 (a) 비트코인 가격 데이터 읽기

```
[18]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

coindesk_data_train = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/traindata.csv", header=0)
coindesk_data_test = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/testdata.csv", header=0)
```

```
coindesk_data_train.head(5)
```

```
[19]:
```

	date	closing price	open price	high price	low price
0	2019-02-28	3816.6	3814.6	3883.7	3783.3
1	2019-03-01	3821.9	3816.7	3855.8	3816.4
2	2019-03-02	3823.1	3821.9	3843.2	3783.6
3	2019-03-03	3809.5	3823.2	3836.6	3789.7
4	2019-03-04	3715.9	3809.7	3828.4	3681.8

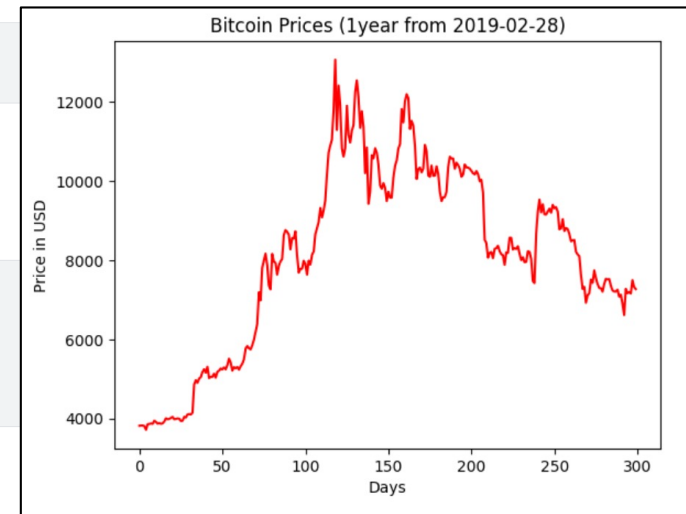
← 종가, 시가, 최고가, 최저가

+ Code + Markdown

```
[20]: seq = coindesk_data_train[['closing price']].to_numpy()
print('데이터 길이:', len(seq), '\n앞쪽 5개 값:', seq[0:5])
```

```
데이터 길이: 300
앞쪽 5개 값: [[3816.6]
 [3821.9]
 [3823.1]
 [3809.5]
 [3715.9]]
```

```
[22]: # 그래프로 데이터 확인
plt.plot(seq, color='red')
plt.title('Bitcoin Prices (1year from 2019-02-28)')
plt.xlabel('Days')
plt.ylabel('Price in USD')
plt.show()
```



8.1.3 시계열 데이터 사례: 비트코인 가격

■ 프로그램 8-1 (a) 비트코인 가격 데이터 읽기

```
[18]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

coindesk_data_train = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/traindata.csv", header=0)
coindesk_data_test = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/testdata.csv", header=0)
```

```
coindesk_data_train.head(5)
```

	date	closing price	open price	high price	low price
0	2019-02-28	3816.6	3814.6	3883.7	3783.3
1	2019-03-01	3821.9	3816.7	3855.8	3816.4
2	2019-03-02	3823.1	3821.9	3843.2	3783.6
3	2019-03-03	3809.5	3823.2	3836.6	3789.7
4	2019-03-04	3715.9	3809.7	3828.4	3681.8

+ Code + Markdown

```
[20]: seq = coindesk_data_train[['closing price']].to_numpy()
print('데이터 길이:', len(seq), '\n앞쪽 5개 값:', seq[0:5])
```

```
데이터 길이: 300
앞쪽 5개 값: [[3816.6]
[3821.9]
[3823.1]
[3809.5]
[3715.9]]
```

```
[22]: # 그래프로 데이터 확인
plt.plot(seq, color='red')
plt.title('Bitcoin Prices (1year from 2019-02-28)')
plt.xlabel('Days')
plt.ylabel('Price in USD')
plt.show()
```

NOTE []와 []의 차이

[프로그램 8-1(a)]에서 08행의 coindesk_data[['Closing Price (USD)']] 구문에 []를 사용했다. 08행을 coindesk_data['Closing Price (USD)']로 바꾸어 [] 방식을 사용하면 무슨 차이가 있을까? 간단한 예를 들면, [] 방식은 [3] [5] [7] [6]과 같이 표현되고 [] 방식은 [3 5 7 6]으로 표현된다. [프로그램 8-1(a)]는 매일 형성되는 네 가지 주식 시세인 종가, 시가, 고가, 저가 중에서 종가만 취하여 실험을 하기 때문에 벡터의 벡터인 []와 벡터인 []의 차이가 두드러지지 않는다. 네 값을 모두 사용하는 다중 채널의 경우는 반드시 벡터의 벡터인 [] 표현을 사용해야 한다. 종가만 사용하는 프로그램에서 [] 표현으로 코딩해두면 다중 채널로 확장이 쉬워지기 때문에 더 좋은 프로그램이 된다. 다중 채널로 확장하는 작업은 [프로그램 8-3]에서 다룬다.

[][] vs []의 차이!

```
seq2 = coindesk_data_train[['closing price']].to_numpy()
```

seq2

```
array([[ 3816.6,  3821.9,  3823.1,  3809.5,  3715.9,  3857.2,  3863. ,
        3875.1,  3865.9,  3944.3,  3915.2,  3870.3,  3886. ,  3865.1,
        3879. ,  3924.3,  4006.4,  3981.5,  3990.2,  4017. ,  4041.2,
        3982.2,  3990.4,  4002.5,  3994.7,  3937. ,  3942.8,  4041.7,
        4025.6,  4102.2,  4111.8,  4102.3,  4145.1,  4859.3,  4968.7,
        4902.4,  5010.2,  5046.2,  5173.6,  5245.2,  5158.4,  5307.8,
        5022.6,  5054.2,  5051.8,  5134.8,  5032.3,  5180.9,  5208.3,
        5264.7,  5241. ,  5290.2,  5243.5,  5346.7,  5511.6,  5415.6,
        5209.1,  5298.3,  5265.9,  5302.3,  5235. ,  5320.8,  5384.2,
        5493.8,  5766.8,  5830.9,  5774.9,  5745.1,  5849.5,  5990.3,
        6191.5,  6386. ,  7190.3,  6984.8,  7886. ,  7994.6,  8164.6,
        7871.8,  7359.5,  7262.6,  8157.2,  7965.3,  7930.3,  7635.7,
        7847.1,  7470.1,  8077.4,  8670.2,  8760.1,  8716.3,  8647.8])
```

```
seq2 = coindesk_data_train[['closing price', 'open price']].to_numpy()
```

seq2

```
array([[ 3816.6,  3814.6],
       [ 3821.9,  3816.7],
       [ 3823.1,  3821.9],
       [ 3809.5,  3823.2],
       [ 3715.9,  3809.7],
       [ 3857.2,  3715.9],
       [ 3863. ,  3857.2],
       [ 3875.1,  3863.1],
       [ 3865.9,  3875.1],
       [ 3944.3,  3865.6],
       [ 3915.2,  3944.4],
       [ 3870.3,  3915.2],
       [ 3886. ,  3870.3],
       [ 3865.1,  3885.9],
```


8.1.3 시계열 데이터 사례: 비트코인 가격

■ 프로그램 8-2(b) 1년치 비트코인 가격 데이터를 윈도우로 자르기

시계열 데이터 전처리

- 이제 시계열 데이터를 윈도우 단위로 나누고, 수평계수를 반영하여 샘플링 해야함

```
[18]: seq = coindesk_data_train[['closing price']].to_numpy()
      print('데이터 길이:', len(seq), '\n앞쪽 5개 값:', seq[0:5])

데이터 길이: 300
앞쪽 5개 값: [[3816.6]
 [3821.9]
 [3823.1]
 [3809.5]
 [3715.9]]

+ Code + Markdown

[18]: def seq2dataset(seq, window, horizon):
      X = []; Y=[]
      for i in range(len(seq)-(window+horizon)+1):
          x=seq[i:(i+window)]
          y=(seq[i+window+horizon-1])
          X.append(x)
          Y.append(y)
      return np.array(X), np.array(Y)

[19]: # W = 이전데이터 수, h = 수평선계수
      w=7; h=1;

      X, Y = seq2dataset(seq,w,h)

      print(X[0],Y[0])

[[3816.6]
 [3821.9]
 [3823.1]
 [3809.5]
 [3715.9]
 [3857.2]
 [3863. ]] [3875.1]
```

	closing price	open price	high price	low price
date				
2019- 02- 28	3816.6	3814.6	3883.7	3783.3
2019- 03- 01	3821.9	3816.7	3855.8	3816.4
2019- 03- 02	3823.1	3821.9	3843.2	3783.6
2019- 03- 03	3809.5	3823.2	3836.6	3789.7
2019- 03- 04	3715.9	3809.7	3828.4	3681.8
2019- 03- 05	3857.2	3715.9	3873.2	3705.7
2019- 03- 06	3863.0	3857.2	3887.3	3816.7
2019- 03- 07	3875.1	3863.1	3907.4	3847.9
2019- 03- 08	3865.9	3875.1	3929.0	3810.7
2019- 03- 09	3944.3	3865.6	3964.0	3859.7

8.2 순환 신경망

- 시계열 데이터를 처리하는 신경망을 설계할 때는 시간에 따라 값이 하나씩 순차적으로 들어온다는 사실을 고려해야 함
- 컨볼루션 신경망(CNN)과 깊은 다층 퍼셉트론(MLP)에서는 데이터가 맨 왼쪽에 있는 입력층으로 한꺼번에 들어온다는 사실을 감안하면, 순환 신경망(RNN)은 이전 신경망과 구조 측면에서 본질적으로 달라야 할 것임
- 순환 신경망(RNN)은 다행스럽게도 다층 퍼셉트론(MLP)을 약간 고쳐서 사용 가능함
 - [주의] 개념 설명을 위해 도입된 것임. 완전 일치하지는 않음

8.2.1 구조와 동작, 학습 알고리즘

- 시계열 데이터를 처리하는 신경망
 - [그림8-5(a)]는 [그림4-13]을 다시 그린 것으로, 순환 신경망(RNN, Recurrent Neural Network)과 비교하기 위해 왼쪽에서 오른쪽으로 데이터가 흐르던 방식을 아래에서 위로 흐르게 90도 회전해 그렸음
 - 또한 U^1 과 U^2 로 표기하던 가중치 집합을 U 와 V 로 바꾸어 표기함

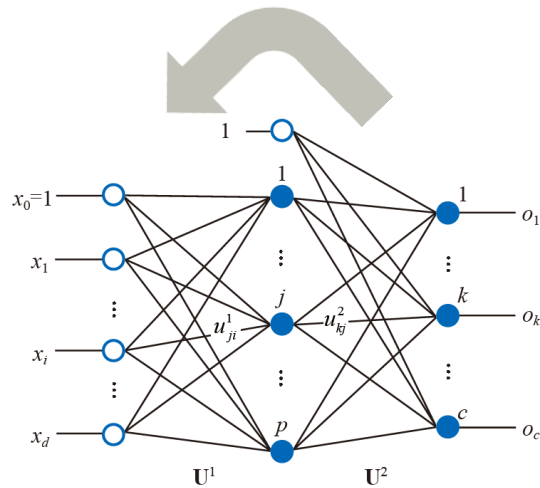
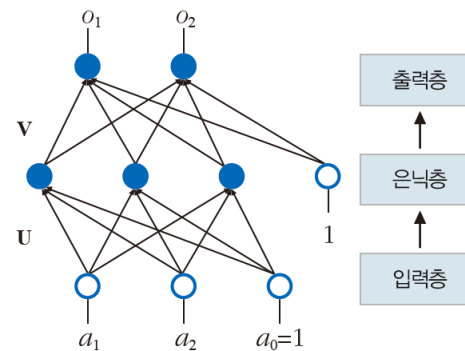
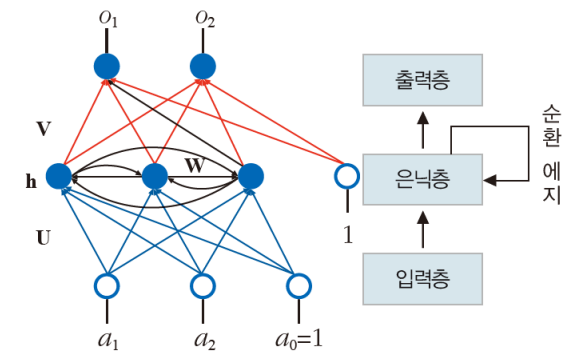


그림 4-13 다층 퍼셉트론의 구조



(a) 다층 퍼셉트론([그림 4-13])

그림 8-5 다층 퍼셉트론과 순환 신경망의 비교

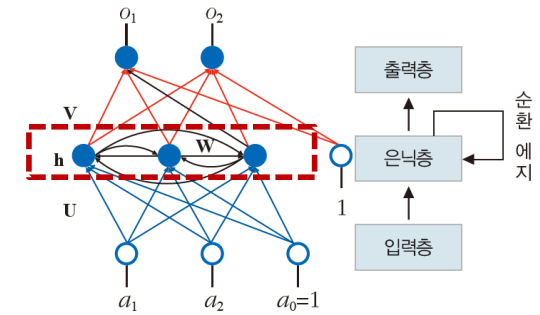


(b) 순환 신경망([그림 4-13]에 순환 에지 추가)

8.2.1 구조와 동작, 학습 알고리즘

■ 시계열 데이터를 처리하는 신경망

- 순환 신경망은 다층 퍼셉트론과 한 가지만 다름
- 순환 신경망에는 은닉층에 있는 노드끼리 엣지로 연결되어 있음
- 이 엣지는 은닉층 내에서 정보를 순환하므로 순환 엣지(recurrent edge)라고 함
- “순환 엣지”라는 단순한 아이디어로 <다층 퍼셉트론(MLP)>이 시계열 데이터 처리에 적합한 <순환 신경망(RNN)>으로 변신한 것임



- 순환 신경망은 순환 엣지를 통해 시간 정보를 입력하고 가변 길이를 다루고 문맥 정보를 처리하는 능력을 갖출 수 있음
 - 시간 정보를 입력 : 데이터가 시계열 순서대로 입력
 - 가변 길이를 다룸 : 입력 길이 만큼 RNN 반복 (입력 길이를 미리 알 필요 없음)
 - 문맥 정보 처리 : 이전 은닉 상태를 현재 은닉 상태에 반영

8.2.1 구조와 동작, 학습 알고리즘

- 시계열 데이터를 처리하는 신경망
 - 순환 에지는 어떻게 시간 정보를 처리할 수 있을까?
 - 아래 [그림 8-6]은 [그림8-5(b)]의 순환 신경망을 단지 펼친 것
 - [그림8-6]의 신경망에는 1이라는 순간, 2라는 순간, ..., t라는 순간이 있음
 - i 순간에는 입력층 a^i 가 입력되고, 은닉층의 상태가 h^i 로 변하고 출력층에서 o^i 가 출력됨
 - [그림 8-5(b)]의 a_0, a_1, a_2 가 [그림 8-6]의 a^i 를 구성

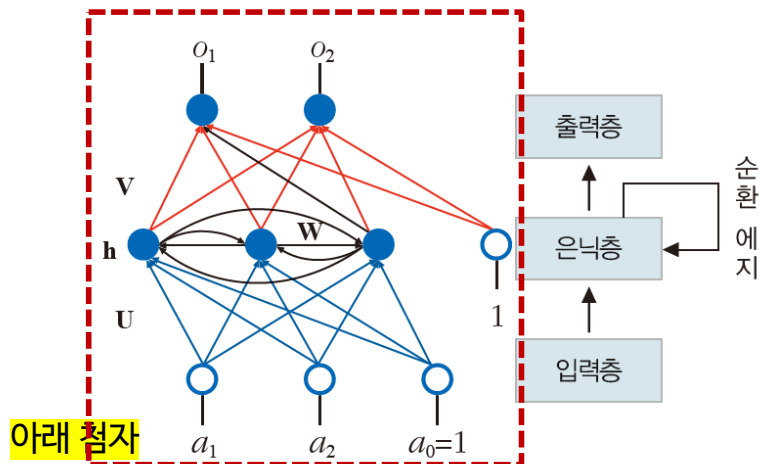


그림 8-5 (b) 순환 신경망(그림 4-13)에 순환 에지 추가

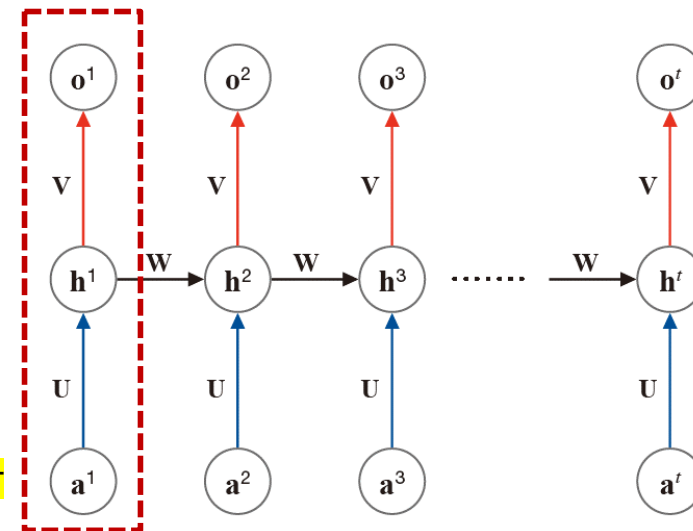
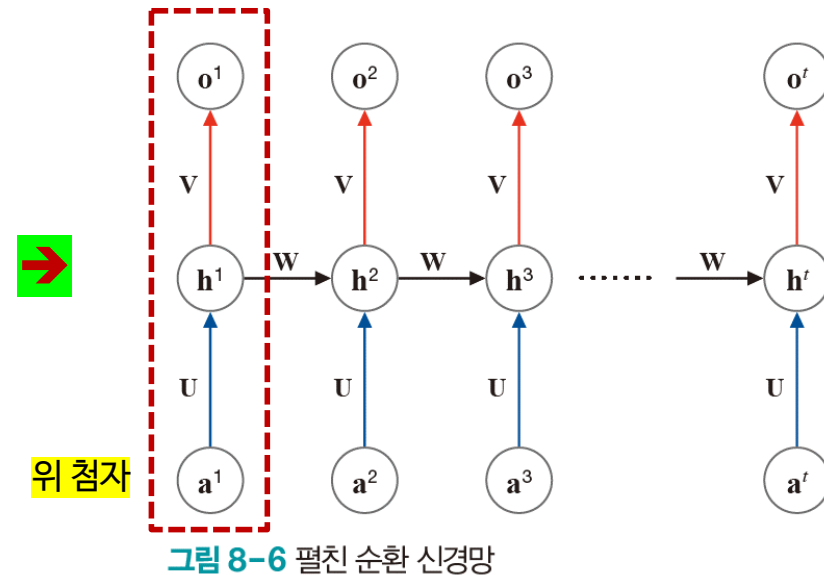
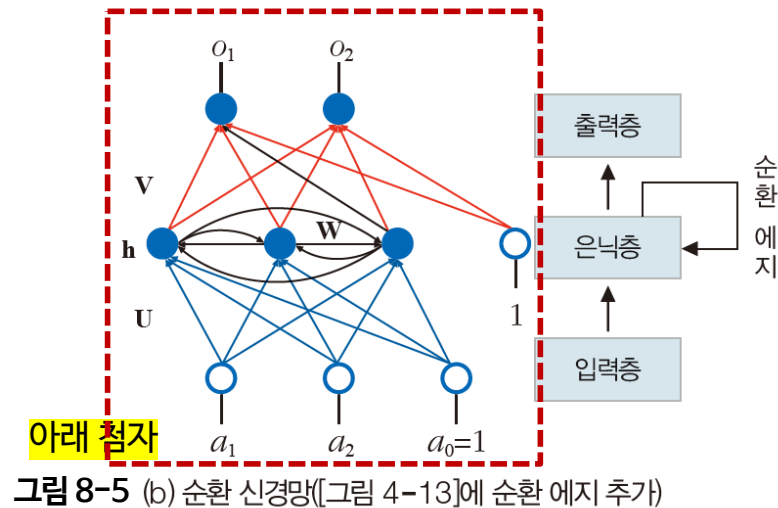


그림 8-6 펼친 순환 신경망

8.2.1 구조와 동작, 학습 알고리즘

- 시계열 데이터를 처리하는 신경망
 - 순환 신경망이 다층 퍼셉트론과 두드러지게 다른 점은 **가중치 집합**임
 - 다층 퍼셉트론 $\{U, V\}$ vs. 순환 신경망 $\{U, V, W\}$
 - 가중치 U, V, W 를 보면 시간을 나타내는 첨자가 없으며, 이는 **순환 신경망이 모든 순간 가중치를 공유한다는 것을 의미함**



8.2.1 구조와 동작, 학습 알고리즘

- 순환 신경망의 동작

- i 순간의 a^i 는 가중치 U 를 통해 은닉층의 상태 h^i 에 영향을 미치고, h^i 는 가중치 V 를 통해 출력값 o^i 에 영향을 미침. h^{i-1} 는 가중치 W 를 통해 h^i 에 영향을 미침

- 은닉층에서 일어나는 계산

$$\mathbf{h}^i = \tau_1(\mathbf{W}\mathbf{h}^{i-1} + \mathbf{U}\mathbf{a}^i) \quad (8.2)$$

- 출력층에서 일어나는 계산

$$\mathbf{o}^i = \tau_2(\mathbf{V}\mathbf{h}^i) \quad (8.3)$$

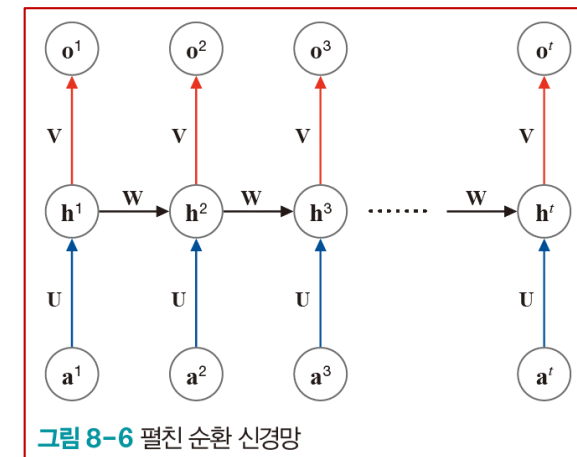
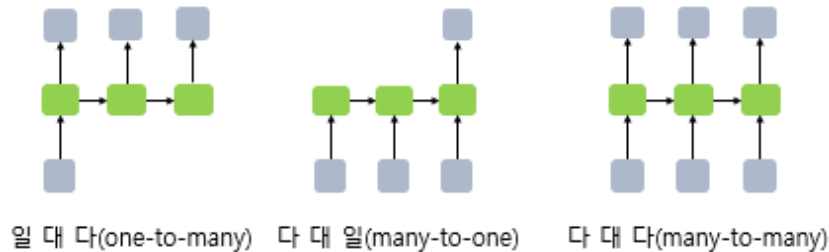


그림 8-6 펼친 순환 신경망

- 식 (8.2)에서 $\mathbf{W}\mathbf{h}^{i-1}$ 항을 제외하면 다층 퍼셉트론과 동일
 - 순환 신경망은 이 항을 통해 이전 순간의 은닉층 상태 h^{i-1} 를 현재 순간의 은닉층 상태 h^i 로 전달하여 시간성을 처리함
 - 순환 신경망의 학습 알고리즘을 BPTT(Back-propagation Through Time)라고 하며, 기존 신경망들의 back-propagation과는 다름

8.2.1 구조와 동작, 학습 알고리즘

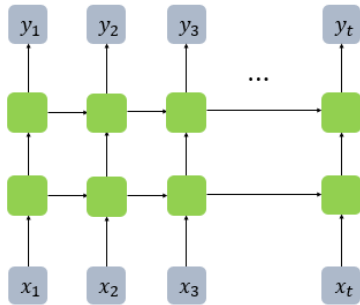
- RNN은 입력과 출력의 길이를 다르게 설계할 수 있으므로 다양한 용도로 사용 가능함



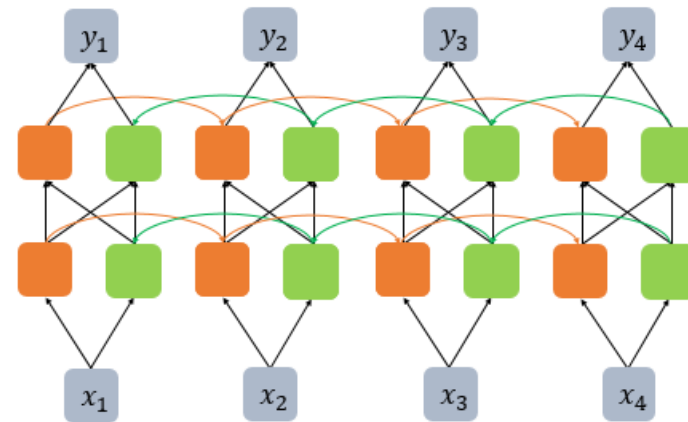
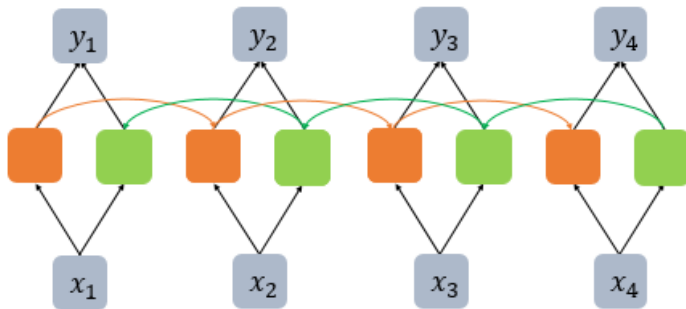
- 일 대 다(one-to-many) : 하나의 이미지 입력에 대해 사진의 제목을 출력하는 이미지 캡셔닝 작업에 사용
- 다 대 일(many-to-one) : 단어의 시퀀스에 대해서 하나의 출력을 하는 스팸 분류에 사용
- 다 대 다(many-to-many) : 입력 문장으로부터 대답 문장을 출력하는 챗봇, 번역기 등

8.2.1 구조와 동작, 학습 알고리즘

- RNN이 다수의 은닉층을 가지는 경우를 깊은 순환 신경망(Deep RNN) 이라고 함



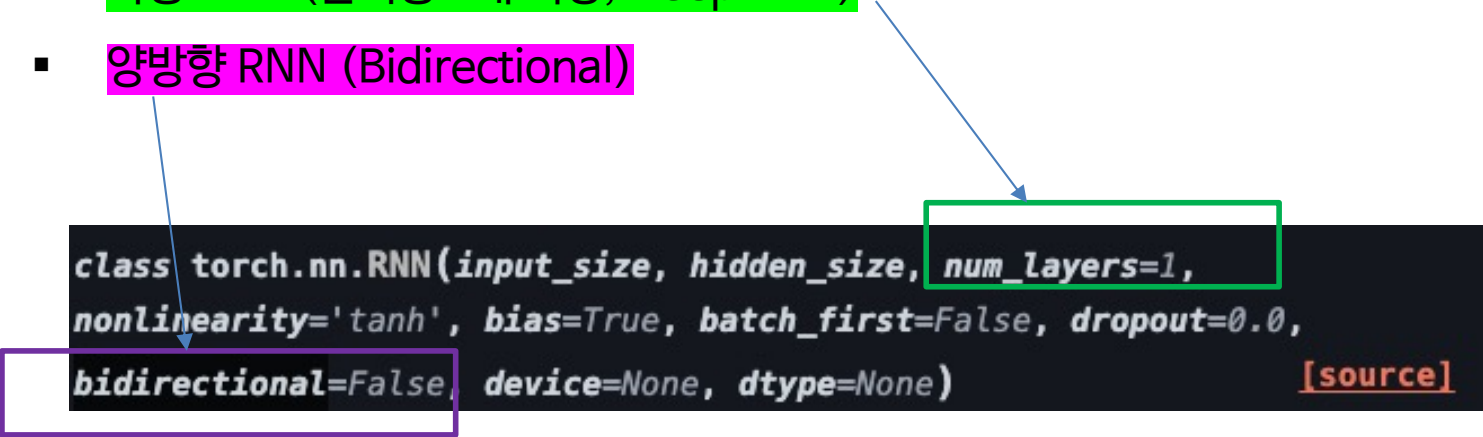
- 양방향 순환 신경망(Bidirectional RNN)은 시점 t 에서의 출력 값을 예측할 때, 이전 시점의 입력 뿐만 아니라, 이후 시점의 입력 또한 예측에 기여할 수 있다는 아이디어에서 기반함



8.2.1 구조와 동작, 학습 알고리즘

- RNN 파이토치 실습
 - 단층 단방향 기본 RNN (은닉층 1개, Basic RNN)
 - 일대다(one-to-many)
 - 다대일(many-to-one)
 - 다층 RNN (은닉층 2개 이상, Deep RNN)
 - 양방향 RNN (Bidirectional)

```
class torch.nn.RNN(input_size, hidden_size, num_layers=1,  
nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0,  
bidirectional=False, device=None, dtype=None) \[source\]
```



8.2.1 구조와 동작, 학습 알고리즘

- 다대일(many-to-one)

```
4 class ManyToOneRNN(nn.Module):
5     """
6     예: 문장(토큰 시퀀스)을 입력 받아 감정 3가지(긍정/부정/중립)로 분류한다고 가정
7     입력: (batch, seq_len, input_size)
8     출력: (batch, num_classes)
9     """
10    def __init__(self, input_size, hidden_size, num_layers, num_classes):
11        super().__init__()
12        self.rnn = nn.RNN(
13            input_size=input_size,
14            hidden_size=hidden_size,
15            num_layers=num_layers,
16            batch_first=True,
17        )
18        self.fc = nn.Linear(hidden_size, num_classes)
19
20    def forward(self, x):
21        # x: (batch, seq_len, input_size)
22        out, h_n = self.rnn(x) # out: (batch, seq_len, hidden_size)
23        last_hidden = out[:, -1] # 마지막 타임스텝만 사용 (batch, hidden_size)
24        logits = self.fc(last_hidden) # (batch, num_classes)
25        return logits
26
```

8.2.1 구조와 동작, 학습 알고리즘

- 다대 일(many-to-one)
 - 단층 RNN - 다대 일 구조 : num_layers = 1
 - 깊은 RNN - 다대 일 구조 : num_layers = 2

```
28 # ---- 사용 예시 ----
29 batch_size = 4
30 seq_len = 10 # 문장 길이 10 토큰
31 input_size = 16 # 각 토큰 임베딩 차원
32 hidden_size = 32
33 num_layers = 1
34 num_classes = 3 # 감정 3종류
35
36 model = ManyToOneRNN(input_size, hidden_size, num_layers, num_classes)
37
38 x = torch.randn(batch_size, seq_len, input_size)
39 logits = model(x)
40
41 print("Many-to-One 출력 shape:", logits.shape) # (4, 3)
42
```

Many-to-One 출력 shape: torch.Size([4, 3])

8.2.1 구조와 동작, 학습 알고리즘

- 다대다(many-to-many)

```
4 class ManyToManyRNN(nn.Module):
5     """
6     예: 입력 시퀀스 각 토큰마다 태그를 붙이는 POS 태깅/NER 등
7     입력: (batch, seq_len, input_size)
8     출력: (batch, seq_len, num_tags)
9     """
10    def __init__(self, input_size, hidden_size, num_layers, num_tags):
11        super().__init__()
12        self.rnn = nn.RNN(
13            input_size=input_size,
14            hidden_size=hidden_size,
15            num_layers=num_layers,
16            batch_first=True,
17        )
18        self.fc = nn.Linear(hidden_size, num_tags)
19
20    def forward(self, x):
21        # x: (batch, seq_len, input_size)
22        out, h_n = self.rnn(x)          # out: (batch, seq_len, hidden_size)
23        tag_logits = self.fc(out)       # (batch, seq_len, num_tags)
24        return tag_logits
25
26
```

8.2.1 구조와 동작, 학습 알고리즘

- 다대다(many-to-many)
 - 단층 RNN - 다대 일 구조 : num_layers = 1
 - 깊은 RNN - 다대 일 구조 : num_layers = 2

```
27 # ---- 사용 예시 ----
28 batch_size = 4
29 seq_len = 10
30 input_size = 16
31 hidden_size = 32
32 num_layers = 1
33 num_tags = 5 # 예: 태그 5종류
34
35 model = ManyToManyRNN(input_size, hidden_size, num_layers, num_tags)
36
37 x = torch.randn(batch_size, seq_len, input_size)
38 tag_logits = model(x)
39
40 print("Many-to-Many 출력 shape:", tag_logits.shape) # (4, 10, 5)
41
```

Many-to-Many 출력 shape: torch.Size([4, 10, 5])

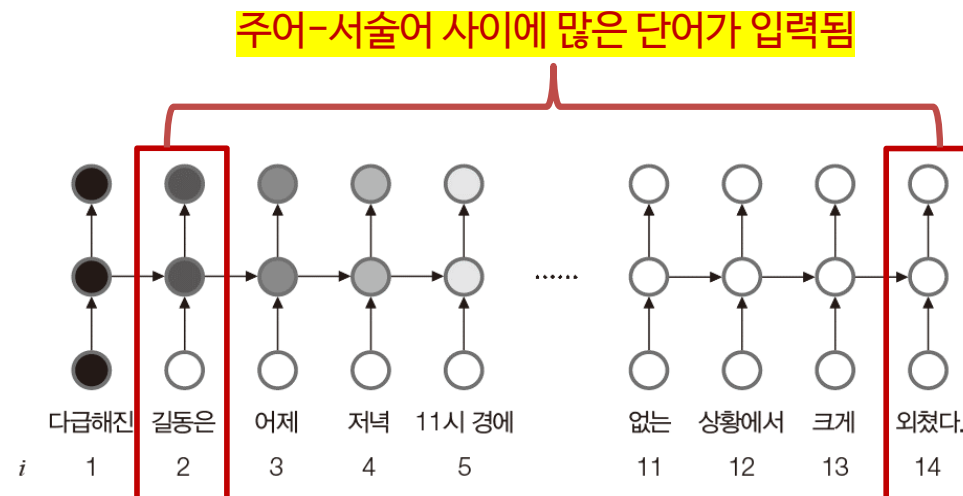


여기까지



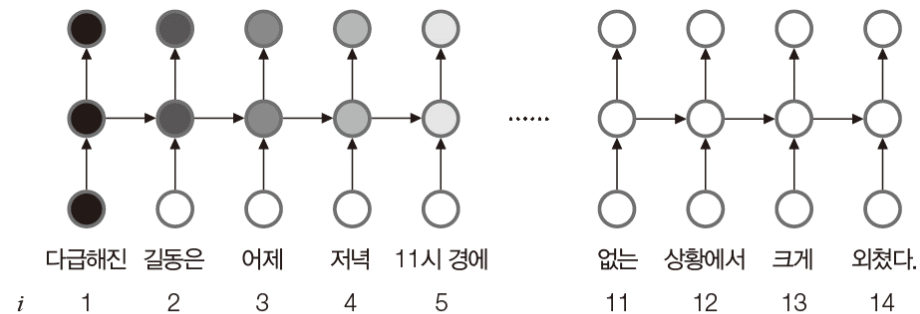
8.2.2 선별 기억력을 갖춘 LSTM

- 순환 신경망(RNN)은 이전 순간의 은닉층 상태를 다음 순간으로 넘기는 기능을 통해 과거를 기억하는 능력이 있으나, 장기 문맥 의존성을 제대로 처리하지 못하는 단점이 있음
- 장기 문맥 의존성 (long-term context dependency)은 떨어져 있는 두 요소가 밀접한 상호작용을 하는 성질을 말하며, 많은 시계열 데이터가 장기 문맥 의존성을 가짐
- 장기 문맥 의존성을 제대로 처리하지 못하는 이유는 계속 들어오는 입력의 영향으로 기억력이 감퇴하는 것이며, 이는 시간이 지남에 따라 기억이 점점 희미해지는 사람 뇌에 비유할 수 있음



8.2.2 선별 기억력을 갖춘 LSTM

- 순환 신경망(RNN)은 이전 순간의 은닉층 상태를 다음 순간으로 넘기는 기능을 통해 과거를 기억하는 능력이 있으나, 장기 문맥 의존성을 제대로 처리하지 못하는 단점이 있음

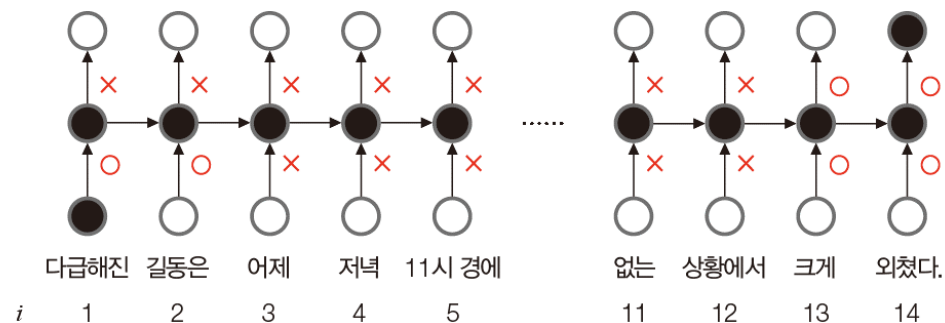


(a) 선별 기억 능력이 없는 순환 신경망

t=2 에 입력된 길동은”이라는 단어는 t=3,4,5 에는 기억이 남아 있지만, 이후에는 거의 사라지는 현상이 나타남

8.2.2 선별 기억력을 갖춘 LSTM

- 사람은 선별 기억(selective memory) 능력이 있어 인상적인 일은 아주 오래 전 기억도 간직함
- LSTM(long short-term memory)은 순환 신경망에 선별 기억 능력을 추가한 신경망임
- LSTM은 게이트(gate)라는 개념을 통해 선별 기억 능력을 확보함



(b) 선별 기억 능력을 지닌 LSTM

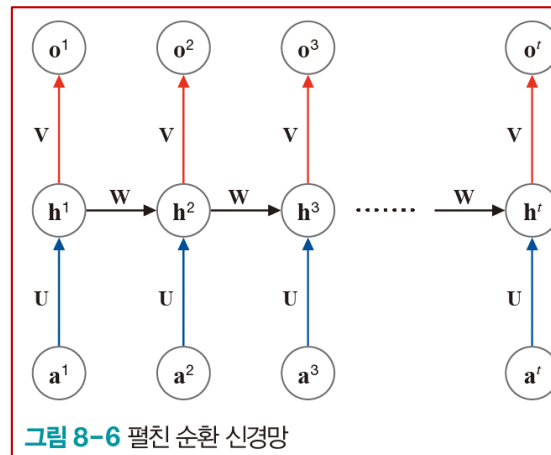
그림 8-7 순환 신경망과 LSTM

그림 8-7(b)는 순환신경망에 빨간 원으로 표시한 게이트를 추가한 LSTM임

O는 열린 상태의 게이트로서 신호를 전송하고, X는 닫힌 상태의 게이트로서 신호를 차단함

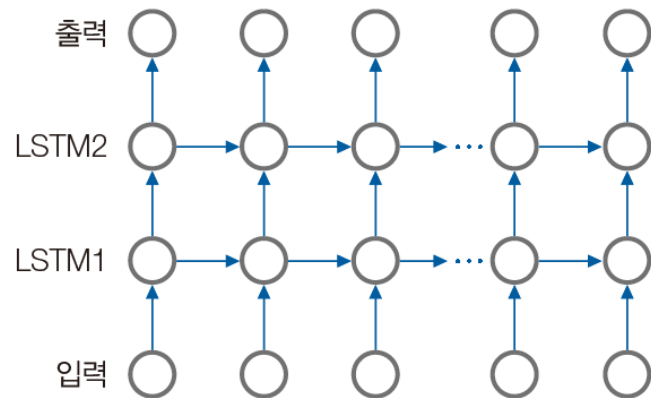
8.2.2 선별 기억력을 갖춘 LSTM

- [그림8-7(b)]는 LSTM을 아주 간단화한 설명임
- 실제로는 열거나 닫는 두 가지 상태만 있는 것이 아니라 여닫는 정도를 조절할 수 있도록 게이트는 0~1사이의 실수 값을 가짐
- 게이트의 여닫는 정도는 LSTM의 가중치로 표현되며 가중치 값은 학습으로 알아냄
- LSTM의 가중치는 순환 신경망의 $\{U, V, W\}$ 에 4개를 추가하여 $\{U, U^i, U^o, W, W^i, W^o, V\}$ 에 해당되며, i 는 입력 게이트, o 는 출력 게이트를 의미함

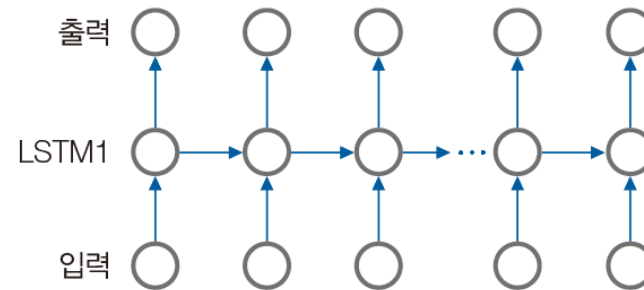


8.2.3 LSTM의 유연한 구조

- 순환 신경망은 다양하게 변형할 수 있음 (단층 → 적층)
- [그림 8-8(a)]는 은닉층을 2개 쌓은 적층 LSTM (stacked LSTM) 임
- 적층 LSTM은 층을 깊게 쌓는 형태이며, 복잡도가 높아 효율적인 특징을 지닌다.



(a) 적층 LSTM



(a') 단층 LSTM

8.2.3 LSTM의 유연한 구조

- 순환 신경망은 다양하게 변형할 수 있음 (단방향 → 양방향)
- 시계열 데이터 중에는 오른쪽 방향과 왼쪽 방향을 모두 살펴야 문맥을 제대로 파악할 수 있는 경우가 많음
- 예) “잘 **달리는** 이 **차**는 유럽에서 생산했다” vs “고산 지대에서 생산한 이 **차**는 **향**이 좋다”
- [그림 8-8(b)]는 첫 번째 은닉층에서는 원래처럼 오른쪽으로 데이터가 흐르고 두 번째 은닉층에서는 데이터가 왼쪽으로 흐름. 이런 신경망을 **양방향 LSTM (bi-directional LSTM)**이라고 함

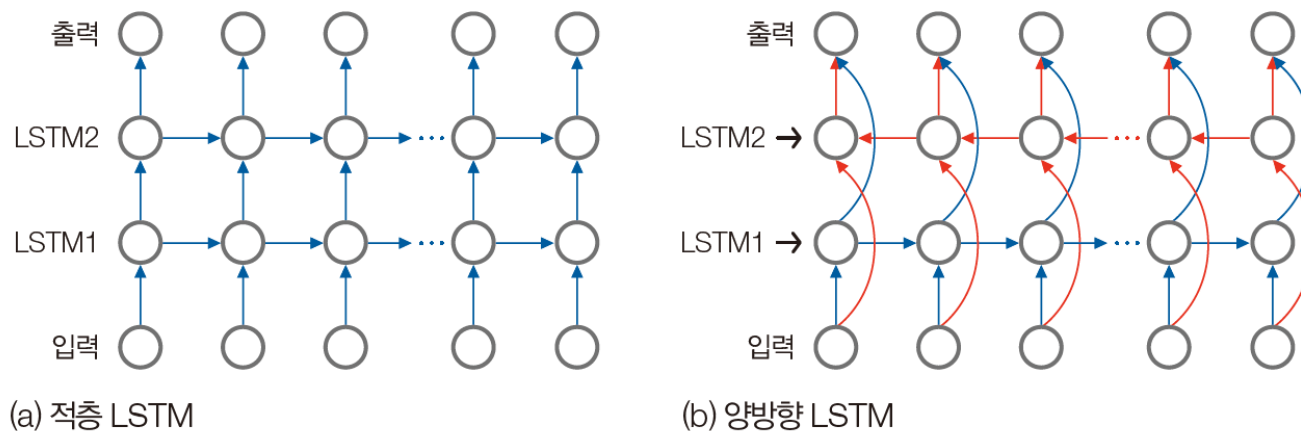


그림 8-8 적층 LSTM과 양방향 LSTM

8.2.3 LSTM의 유연한 구조

- [그림 8-9]는 여러 응용 문제를 위한 다양한 LSTM 신경망 구조를 보여줌
 - [그림 8-9(a)] 미래 예측, 비트코인 가격 예측, 심전도 데이터를 통한 환자 예측 문제에 적합한 신경망 구조
 - [그림 8-9(b)] 비디오를 구성하는 프레임별 영상 내용 분류 응용에 적합한 신경망 구조
 - [그림 8-9(c)] 언어 번역 응용에 적합한 신경망 구조
 - 앞부분에 한국어 문장을 구성하는 단어가 순서대로 입력되면 구문과 의미를 파악하고, 뒤에서는 영어 문장을 생성해 출력하는 방식

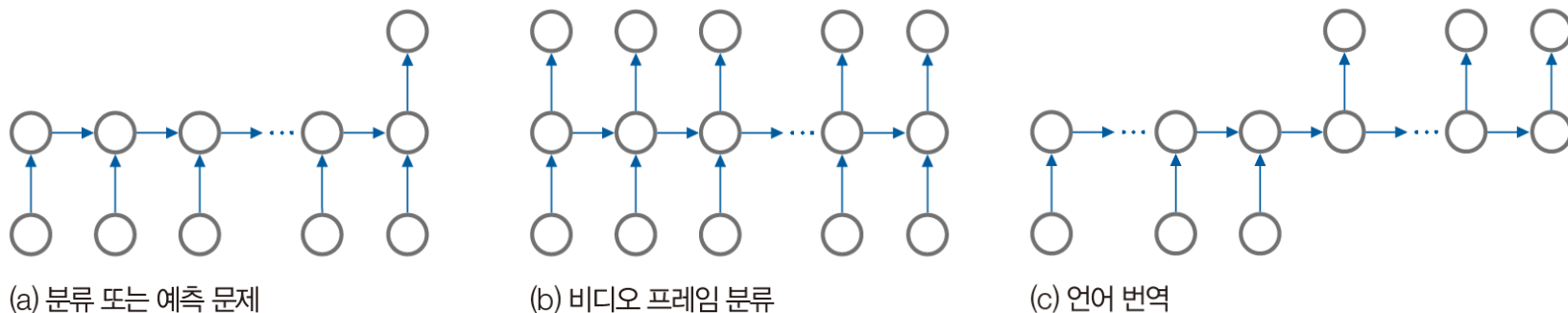


그림 8-9 다양한 문제에 적용할 수 있는 LSTM

8.3 LSTM으로 시계열 예측하기

- 이 절에서는 시계열 데이터를 보고 미래를 예측하는 프로그래밍 실습을 진행함
- [프로그램 8-1]에서 다룬 비트코인 가격 데이터를 가지고 LSTM을 학습하며, 성능 측정 기준에 따라 LSTM의 예측 능력을 평가함
- (1) 종가 정보만 있는 **단일 채널 시계열 데이터**에 적용
- (2) (종가, 시가, 고가, 저가)의 **다중 채널 시계열 데이터**에 적용

8.3.1 단일 채널 비트코인 가격 예측

- [그림 8-4(b)]를 보면 비트코인 가격 데이터에는 (종가, 시가, 고가, 저가)의 네 열이 있으며, 해당 데이터는 입력으로 모두 사용될 수 있음
- 하지만, 프로그램 8-2에서는 종가만을 사용하여 입력이 1개인 단일 채널 데이터를 다루기로 함



(a) 코인데스크에서 데이터를 다운로드하기

	A	B	C	D	E	F	G
1	Currency	Date	Closing Price	24h Open	24h High	24h Low	(USD)
2	BTC	2019-02-28	3772.94	3796.64	3824.17	3666.52	
3	BTC	2019-03-01	3799.68	3773.44	3879.23	3753.8	
4	BTC	2019-03-02	3811.61	3799.37	3840.04	3788.92	
5	BTC	2019-03-03	3804.42	3806.69	3819.19	3759.41	
6	BTC	2019-03-04	3782.66	3807.85	3818.7	3766.24	
7	BTC	2019-03-05	3689.86	3783.36	3804.35	3663.48	
8	BTC	2019-03-06	3832.08	3701.05	3866.72	3688.7	
9	BTC	2019-03-07	3848.96	3832.59	3881.97	3802.52	
10	BTC	2019-03-08	3859.84	3848.96	3890.75	3827.67	
11	BTC	2019-03-09	3828.37	3859.8	3918	3778.52	
12	BTC	2019-03-10	3898.87	3841.89	3948.88	3832.14	
13	BTC	2019-03-11	3899.66	3916.76	3921.93	3865.92	
14	BTC	2019-03-12	3851.25	3899.46	3913.46	3819.93	

(b) 비트코인 가격이 저장된 데이터 프레임

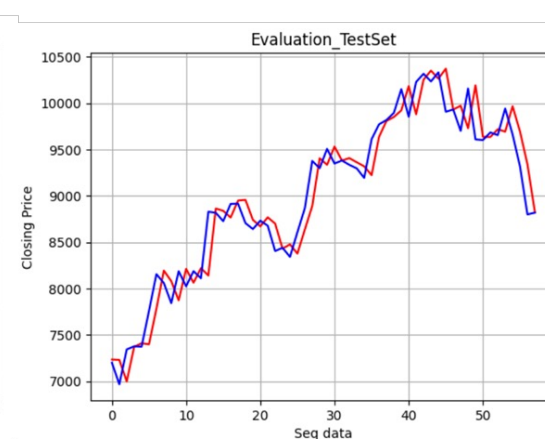
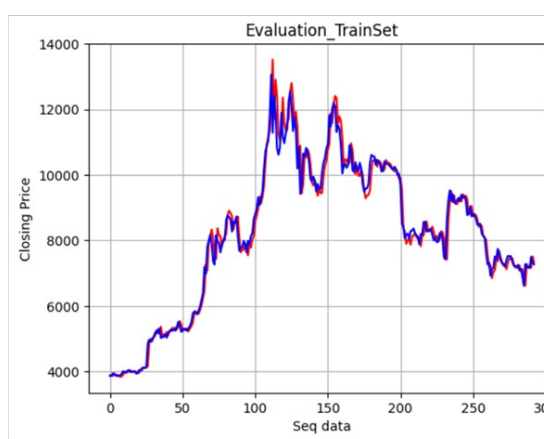
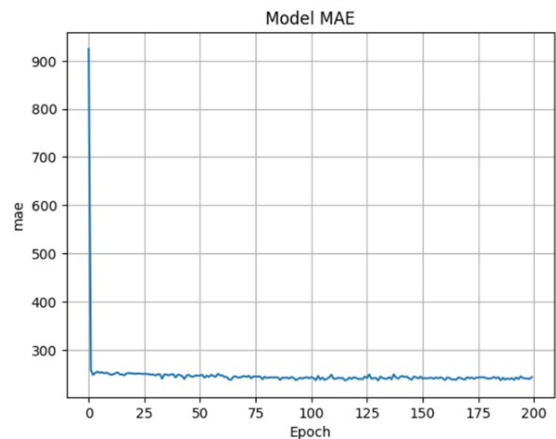
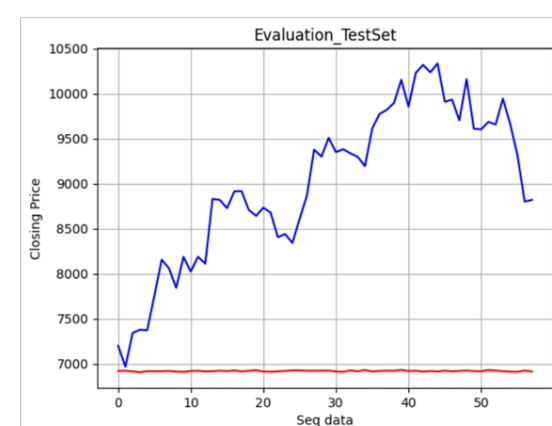
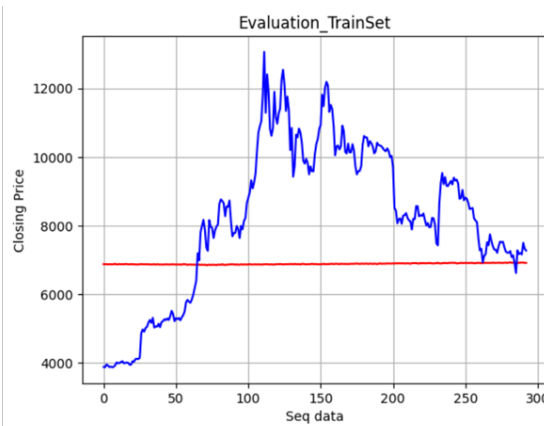
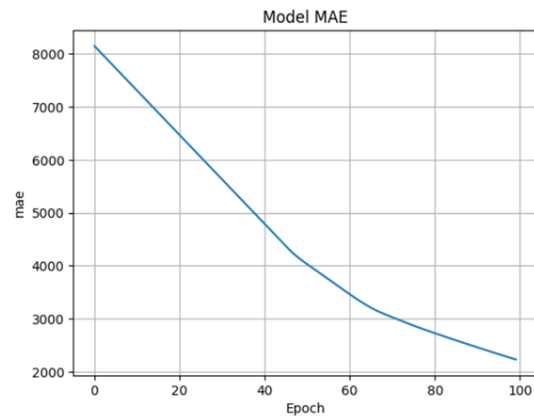
그림 8-4 코인데스크 사이트에서 다운로드한 비트코인 가격 데이터

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 리더보드
 - <https://www.kaggle.com/competitions/2024-1-dl-w-11-p-1-1-btc-price-prediction>
 - (1) 파이토치 내 lstm을 사용하는 솔루션
 - <https://www.kaggle.com/code/yukyungchoi/2024-dl-lstm-1-1>
 - (2) 파이토치 내 lstm의 활성화함수를 relu로 사용하는 솔루션
 - <https://www.kaggle.com/code/yukyungchoi/2024-dl-lstm-1-3>
 - (3) 파이토치 내 lstm을 사용하고 데이터 정규화를 사용하는 솔루션
 - <https://www.kaggle.com/code/yukyungchoi/2024-dl-lstm-1-2>

8.3.1 단일 채널 비트코인 가격 예측

- Pytorch의 LSTM의 경우 내부 활성화함수로 tanh를 사용 중이며, 이를 교안과 같이 ReLU로 커스터마이징할 경우 학습 결과는 아래와 같음



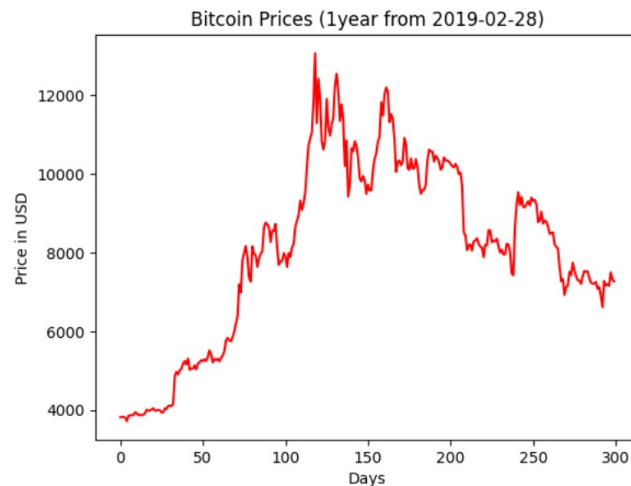
8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측 : **(1) 파이토치 내 lstm을 사용하는 솔루션**
 - 학습 및 테스트 데이터 준비

```
[17]: pd_train = pd.read_csv("/kaggle/input/2024-1-dl-w-11-p-1-1-btc-price-prediction/traindata.csv")
pd_test = pd.read_csv("/kaggle/input/2024-1-dl-w-11-p-1-1-btc-price-prediction/testdata.csv")
```

```
[18]: # 데이터 가공
np_train = pd_train[['closing price']].to_numpy()
np_test = pd_test[['closing price']].to_numpy()

import matplotlib.pyplot as plt
plt.plot(np_train, color='red')
plt.title('Bitcoin Prices (1year from 2019-02-28)')
plt.xlabel('Days')
plt.ylabel('Price in USD')
plt.show()
```



	A	B	C	D	E	F	G
1	Currency	Date	Closing Price	24h Open	24h High	24h Low (USD)	
2	BTC	2019-02-28	3772.94	3796.64	3824.17	3666.52	
3	BTC	2019-03-01	3799.68	3773.44	3879.23	3753.8	
4	BTC	2019-03-02	3811.61	3799.37	3840.04	3788.92	
5	BTC	2019-03-03	3804.42	3806.69	3819.19	3759.41	
6	BTC	2019-03-04	3782.66	3807.85	3818.7	3766.24	
7	BTC	2019-03-05	3689.86	3783.36	3804.35	3663.48	
8	BTC	2019-03-06	3832.08	3701.05	3866.72	3688.7	
9	BTC	2019-03-07	3848.96	3832.59	3881.97	3802.52	
10	BTC	2019-03-08	3859.84	3848.96	3890.75	3827.67	
11	BTC	2019-03-09	3828.37	3859.8	3918	3778.52	
12	BTC	2019-03-10	3898.87	3841.89	3948.88	3832.14	
13	BTC	2019-03-11	3899.66	3916.76	3921.93	3865.92	
14	BTC	2019-03-12	3851.25	3899.46	3913.46	3819.93	

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 윈도우에 맞게 시계열 데이터 준비 ($w = 7, h = 1$)

```
[19]: # 학습데이터와 테스트데이터가 나눠 제공되기 때문에, 교재와 차이가 있음
def seq2dataset(seq, window, horizon, isTrain=True):
    """
    seq: np.array format
    window: sampling step
    """
    X=[]; Y=[]

    for i in range(len(seq)-(window+horizon)+1):
        x=seq[i:(i+window)]
        X.append(x);

        if isTrain:
            y=(seq[i+window+horizon-1])
            Y.append(y)

    if isTrain:
        return np.array(X), np.array(Y)
    else:
        return np.array(X)
```

+ Code + Markdown

```
[20]: # 시계열 데이터 준비

w = 7
h = 1

X_train, Y_train = seq2dataset(np_train, w, h)
X_test = seq2dataset(np_test, w, h, isTrain=False)

print(X_train.shape, Y_train.shape)
print(X_test.shape)

(293, 7, 1) (293, 1)
(58, 7, 1)
```

	closing price	open price	high price	low price
0	3816.6	3814.6	3883.7	3783.3
1	3821.9	3816.7	3855.8	3816.4
2	3823.1	3821.9	3843.2	3783.6
3	3809.5	3823.2	3836.6	3789.7
4	3715.9	3809.7	3828.4	3681.8
5	3857.2	3715.9	3873.2	3705.7
6	3863.0	3857.2	3887.3	3816.7
7	3875.1	3863.1	3907.4	3847.9
8	3865.9	3875.1	3929.0	3810.7
9	3944.3	3865.6	3964.0	3859.7

Train_X

```
array([[3816.6, 3814.6, 3883.7, 3783.3],
       [3821.9, 3816.7, 3855.8, 3816.4],
       [3823.1, 3821.9, 3843.2, 3783.6],
       ...,
       [3715.9, 3809.7, 3828.4, 3681.8],
       [3857.2, 3715.9, 3873.2, 3705.7],
       [3863. , 3857.2, 3887.3, 3816.7]],
      dtype=float64)
```

Train_Y

```
[32... array([[3875.1, 3863.1, 3907.4, 3847.9],
          [3865.9, 3875.1, 3929. , 3810.7],
          [3944.3, 3865.6, 3964. , 3859.7],
          ...,
          [7495.8, 7156.3, 7501.1, 7142. ],
          [7322.8, 7496.2, 7684. , 7276.2],
          [7268.3, 7322.4, 7433.2, 7185.7]])
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 모델 정의

[10]:

```
1 # LSTM 모델 정의 : input 4, hidden 128, output 4
2 # 단일 LSTM 모델 : num_layers = 1
3
4 class LSTMModel(nn.Module):
5     def __init__(self, input_dim, hidden_dim, output_dim, num_layers=1):
6         super().__init__()
7         self.num_layers = num_layers
8         self.hidden_dim = hidden_dim
9         self.lstm = nn.LSTM(input_dim, self.hidden_dim, num_layers=self.num_layers, batch_first=True)
10        self.fc = nn.Linear(self.hidden_dim, output_dim, bias=True)
11
12    def forward(self, x):
13
14        self.h0 = torch.randn(self.num_layers, x.size(0), self.hidden_dim).cuda()
15        self.c0 = torch.randn(self.num_layers, x.size(0), self.hidden_dim).cuda()
16        x, _ = self.lstm(x, (self.h0, self.c0))
17
18        x = x[:, -1, :]
19        x = self.fc(x)
20        return x
```

Num_layers = 2 이상이 되면, stacked LSTM 모델이 됨

+ Code

+ Markdown

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 모델 학습을 위한 데이터 배치형 가공

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, TensorDataset
from torch.utils.data import DataLoader

# 데이터 토치텐서로 변경
train_dataset = TensorDataset(torch.FloatTensor(X_train), torch.FloatTensor(Y_train))
test_dataset = TensorDataset(torch.FloatTensor(X_test))

# 데이터 배치형 데이터로 변경
# DataLoader 파라미터 shuffle은 default 값이 false
train_loader = DataLoader(train_dataset, batch_size=1)
test_loader = DataLoader(test_dataset, batch_size=1)

# 배치형 데이터 확인법
#train_features, train_labels = next(iter(train_loader))
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 모델 학습

```
▷ # 모델 생성
model = LSTMModel(input_dim=1, hidden_dim=128, output_dim=1)
criterion = nn.L1Loss(reduction='mean')
optimizer = optim.Adam(model.parameters(), lr=1e-2, eps=1e-7)

# 학습 준비
epoch = 100
model = model.cuda()

hist = []
display_Y = []

# 학습 루프
for epoch in range(epoch):
    # 데이터 준비
    avg_loss = 0
    model.train()
    for idx, data in enumerate(train_loader):

        train_X = data[0].cuda()
        train_Y = data[1].cuda()

        # 모델 예측
        pred_Y = model(train_X)

        # 예측 값 시각화를 위한 저장
        if epoch == 99:
            display_Y.append(pred_Y.cpu().detach().numpy())

        # 손실 계산 및 역전파
        loss = criterion(pred_Y, train_Y)
        optimizer.zero_grad()
        loss.backward()
        avg_loss += loss

    # 최적화 수행
    optimizer.step()

    hist.append(avg_loss.cpu().detach().numpy()/len(train_loader))

# 손실 출력 (인터벌 5로 출력)
if epoch%5==0:
    print(f"Epoch {epoch + 1} | Loss: {(avg_loss/len(train_loader)).item():.4f}")
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 학습된 LSTM 모델을 이용한 테스트 데이터 예측

[29]:

```
model.eval()

results = []
with torch.no_grad():
    for idx, data in enumerate(test_loader):
        test_X = data[0].cuda()
        pred_Y = model(test_X)

        # 아래 타입변경과 쌍임
        results.append(pred_Y.cpu())
        #print(test_X[-1].mean(), pred_Y.mean())

# 타입변경
results = np.array(results).reshape(-1,1)
```


8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 캐글 리더보드 제출을 위한 csv 파일 만들기

[30]:

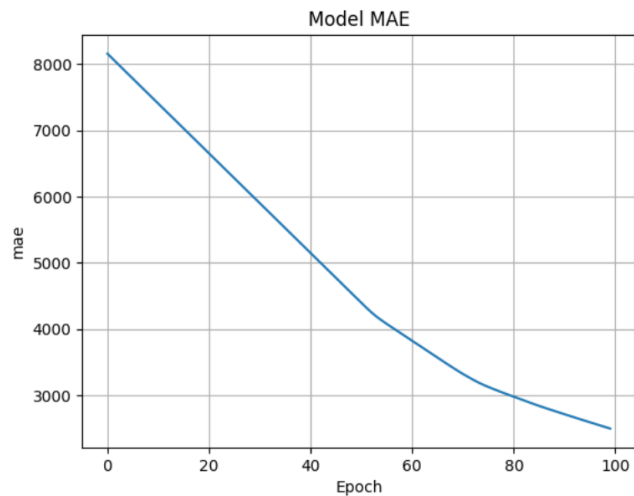
```
# 결과 저장 및 제출 파일 준비
submit = pd.read_csv("/kaggle/input/2024-1-dl-w-11-p-1-1-btc-price-prediction/sample_submit.csv")
submit[['closing price']] = results
submit.to_csv("result.csv", index=False)
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측
 - 시각화

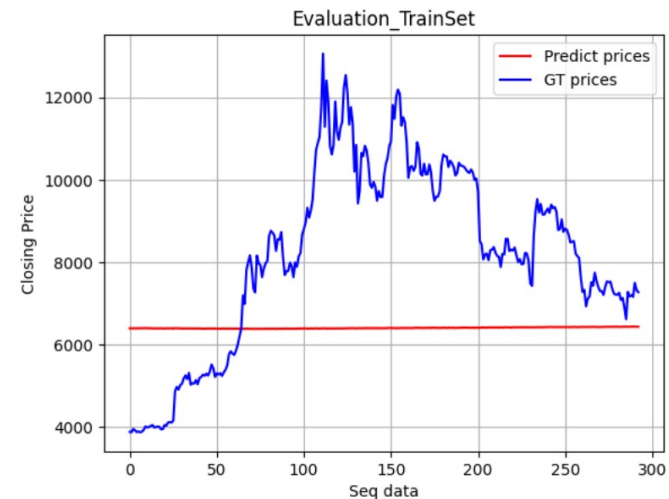
[27]:

```
# 학습 과정 내 손실 값 시각화
plt.plot(hist)
plt.title('Model MAE')
plt.xlabel('Epoch')
plt.ylabel('mae')
plt.grid()
plt.show()
```



[28]:

```
# 학습 데이터 종가과 예측된 종가 시각화
plt.title('Evaluation_TrainSet')
plt.xlabel('Seq data')
plt.ylabel('Closing Price')
plt.plot(np.array(display_Y).reshape(-1), color='red')
plt.plot(np.array(Y_train).reshape(-1), color='blue')
plt.legend(['Predict prices', 'GT prices'], loc='best')
plt.grid()
plt.show()
```



8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측 :
 - (2) 파이토치 내 lstm의 활성화함수를 relu로 사용하는 솔루션
 - <https://www.kaggle.com/code/yukyungchoi/2024-dl-lstm-1-3>
- 모델 학습

```
# LSTM 모델 정의 : input 1, hidden 128, output 1
# 단일 LSTM 모델 : num_layers = 1

class LSTMModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, num_layers=1):
        super().__init__()
        self.num_layers = num_layers
        self.hidden_dim = hidden_dim
        #
        # self.lstm = nn.LSTM(input_dim, self.hidden_dim, num_layers=self.num_layers, batch_first=True)
        self.lstm = CustomLSTMCell(input_dim, self.hidden_dim, self.num_layers, dropout=0, activation='relu')
        self.fc = nn.Linear(self.hidden_dim, output_dim, bias=True)

    def forward(self, x):

        self.h0 = torch.randn(self.num_layers, x.size(0), self.hidden_dim).cuda()
        self.c0 = torch.randn(self.num_layers, x.size(0), self.hidden_dim).cuda()
        x, _ = self.lstm(x, (self.h0, self.c0))

        if len(x.shape)==4:
            x = x.squeeze(0)

        x = x[:, -1, :]
        x = self.fc(x)
        return x
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측 :
 - 모델 학습 (모델 정의)

```
# ReLU 기반 LSTM 모델 정의 : input 1, hidden 128, output 1
# ReLU 기반 단일 LSTM 모델 : num_layers = 1

import torch.nn.functional as F

class CustomLSTMCell(nn.Module):
    def __init__(self, input_size, hidden_size, nlayers, dropout, activation='tanh'):
        """Constructor of the class"""
        super(CustomLSTMCell, self).__init__()

        self.nlayers = nlayers
        self.dropout = nn.Dropout(p=dropout)

        ih, hh = [], []

        for i in range(nlayers):
            ih.append(nn.Linear(input_size, 4 * hidden_size))
            hh.append(nn.Linear(hidden_size, 4 * hidden_size))

        self.w_ih = nn.ModuleList(ih)
        self.w_hh = nn.ModuleList(hh)

        if activation == 'relu':
            self.outputact = F.relu
        elif activation == 'tanh':
            self.outputact = F.tanh
        else:
            assert print("No defined activation function")
```

```
def forward(self, input, hidden):
    """Defines the forward computation of the LSTMCell"""
    hy, cy = [], []

    for i in range(self.nlayers):
        B, L, D = input.shape
        input = input.view(B*L, -1)

        hx, cx = hidden[0][i], hidden[1][i].view(B, 1, -1)

        gates = self.w_ih[i](input).view(B, L, -1) + self.w_hh[i](hx).view(B, 1, -1)
        i_gate, f_gate, c_gate, o_gate = gates.chunk(4, 2)

        i_gate = F.sigmoid(i_gate)
        f_gate = F.sigmoid(f_gate)
        c_gate = self.outputact(c_gate)
        o_gate = F.sigmoid(o_gate)

        ncx = (f_gate * cx) + (i_gate * c_gate)
        nhx = o_gate * self.outputact(ncx)
        cy.append(ncx)
        hy.append(nhx)
        input = self.dropout(nhx)

    hy, cy = torch.stack(hy, 0), torch.stack(cy, 0)

    return hy, cy
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측 :
 - (3) 파이토치 내 lstm을 사용하고 데이터 정규화를 사용하는 솔루션
 - <https://www.kaggle.com/code/yukyungchoi/2024-dl-lstm-1-2>
- 데이터 정규화

```
In [5]: # 데이터 가공
np_train = pd_train[['closing price']].to_numpy()
np_test = pd_test[['closing price']].to_numpy()

# 데이터 전처리
scaler = Scaler()
scaler.set_(X=np_train, ax=0)
np_train = scaler.MinMaxScaler(np_train)
np_test = scaler.MinMaxScaler(np_test)

import matplotlib.pyplot as plt
plt.plot(np_train,color='red')
plt.title('Bitcoin Prices (1year from 2019-02-28)')
plt.xlabel('Days')
plt.ylabel('Price in USD')
plt.show()
```

8.3.1 단일 채널 비트코인 가격 예측

- [프로그램 8-2] 단일 채널 비트코인 가격 예측 :
 - 데이터 정규화

In [4]:

```
class Scaler:
    def __init__(self):
        self.max_ = 0
        self.min_ = 0
        self.ax = 0

    def set_(self, X, ax):
        self.ax = ax
        self.max_ = X.max(self.ax)
        self.min_ = X.min(self.ax)

    def MinMaxScaler(self, X):
        return (X - self.min_) / (self.max_ - self.min_)

    def inverseMinMaxScaler(self, X_pp):
        return (X_pp * (self.max_ - self.min_)) + self.min_
```

8.3.2 성능 평가

- 평균절댓값오차(mean absolute error(MAE)): 스케일 문제에 대처하지 못함
 - 농산물의 판매량 예측에서 복숭아 MAE 1.8 의 결과가 수박 MAE 18 보다 성능이 뛰어나다고 말할 수 없음

$$\text{평균절댓값오차: MAE} = \frac{1}{|M|} \sum_{\mathbf{x} \in M} |y - o| \quad (8.4)$$

- 평균절댓값백분율오차: 스케일 문제에 대처

$$\text{평균절댓값백분율오차: MAPE} = \frac{1}{|M|} \sum_{\mathbf{x} \in M} \left| \frac{y - o}{y} \right| \quad (8.5)$$

표 8-1 평균절댓값오차와 평균절댓값백분율오차

데이터 스케일	데이터(y_test는 참값, pred는 예측값)	평균절댓값오차(MAE)	평균절댓값백분율오차(MAPE)
두 자릿수	y_test [12 20 18 22 28]	(2+1+2+2+2)/5 =1.8	(2/12+1/20+2/18+2/22+2/28)/5 =0.0980
	pred [10 21 16 20 26]		
세 자릿수	y_test [120 200 180 220 280]	(20+10+20+20+20)/5 =18	(20/120+10/200+20/180+20/220+ 20/280)/5=0.0980
	pred [100 210 160 200 260]		

8.3.2 성능 평가

- 등락 정확률
 - 등락을 얼마나 정확하게 맞히는지 측정
 - 맞힌 경우의 수를 전체 샘플 수로 나눔

표 8-2 등락 정확률

샘플 i	x_test[i]	y_test[i]	pred[i]	맞힘
1	[..., ..., ..., 21]	23 업	24 업	o
2	[..., ..., ..., 25]	20 다운	26 업	x
3	[..., ..., ..., 22]	24 업	23 업	o
4	[..., ..., ..., 28]	25 다운	26 다운	o
5	[..., ..., ..., 21]	18 다운	17 다운	o
6	[..., ..., ..., 32]	31 다운	33 업	x
7	[..., ..., ..., 35]	36 업	37 업	o
8	[..., ..., ..., 20]	19 다운	22 업	x
				등락 정확률 = 5/8 = 62.5%

8.3.3 다중 채널 비트코인 가격 예측

- [그림 8-4(b)]를 보면 비트코인 가격 데이터에는 (종가, 시가, 고가, 저가)의 네 열이 있으며, 해당 데이터는 입력으로 모두 사용될 수 있음
- 프로그램 8-3에서는 입력이 4개인 다중 채널 데이터를 다루기로 함



(a) 코인데스크에서 데이터를 다운로드하기

	A	B	C	D	E	F	G
1	Currency	Date	Closing Price	24h Open	24h High	24h Low (USD)	
2	BTC	2019-02-28	3772.94	3796.64	3824.17	3666.52	
3	BTC	2019-03-01	3799.68	3773.44	3879.23	3753.8	
4	BTC	2019-03-02	3811.61	3799.37	3840.04	3788.92	
5	BTC	2019-03-03	3804.42	3806.69	3819.19	3759.41	
6	BTC	2019-03-04	3782.66	3807.85	3818.7	3766.24	
7	BTC	2019-03-05	3689.86	3783.36	3804.35	3663.48	
8	BTC	2019-03-06	3832.08	3701.05	3866.72	3688.7	
9	BTC	2019-03-07	3848.96	3832.59	3881.97	3802.52	
10	BTC	2019-03-08	3859.84	3848.96	3890.75	3827.67	
11	BTC	2019-03-09	3828.37	3859.8	3918	3778.52	
12	BTC	2019-03-10	3898.87	3841.89	3948.88	3832.14	
13	BTC	2019-03-11	3899.66	3916.76	3921.93	3865.92	
14	BTC	2019-03-12	3851.25	3899.46	3913.46	3819.93	

(b) 비트코인 가격이 저장된 데이터 프레임

그림 8-4 코인데스크 사이트에서 다운로드한 비트코인 가격 데이터

8.3.3 다중 채널 비트코인 가격 예측

- [프로그램 8-3] 다중 채널 비트코인 가격 예측
 - 리더보드
 - <https://www.kaggle.com/competitions/2024-1-dl-w-11-btc-price-prediction/overview>
 - 단일 채널 비트코인 가격 예측 문제를 기반으로 해결 하시오

8.3.3 다중 채널 비트코인 가격 예측

- [프로그램 8-3] 다중 채널 비트코인 가격 예측
 - 학습 및 테스트 데이터 준비

```
1 pd_train = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/traindata.csv")
2 pd_test = pd.read_csv("/kaggle/input/2024-1-dl-w-11-btc-price-prediction/testdata.csv")
3
4 #print(pd_train.shape, pd_test.shape)
```

(300, 5) (65, 5)

+ Code

+ Markdown

[19]:

```
1 # 학습 데이터 확인 후 불필요한 정보 삭제
2 pd_train.head(3)
```

+ Code

+ Markdown

[5]:

```
1 pd_train = pd_train.drop('date', axis=1)
2 pd_test = pd_test.drop('date', axis=1)
3 print(pd_train.shape, pd_test.shape)
```

(300, 4) (65, 4)

	A	B	C	D	E	F	G
1	Currency	Date	Closing Pr	24h Open	24h High	24h Low	(USD)
2	BTC	2019-02-28	3772.94	3796.64	3824.17	3666.52	
3	BTC	2019-03-01	3799.68	3773.44	3879.23	3753.8	
4	BTC	2019-03-02	3811.61	3799.37	3840.04	3788.92	
5	BTC	2019-03-03	3804.42	3806.69	3819.19	3759.41	
6	BTC	2019-03-04	3782.66	3807.85	3818.7	3766.24	
7	BTC	2019-03-05	3689.86	3783.36	3804.35	3663.48	
8	BTC	2019-03-06	3832.08	3701.05	3866.72	3688.7	
9	BTC	2019-03-07	3848.96	3832.59	3881.97	3802.52	
10	BTC	2019-03-08	3859.84	3848.96	3890.75	3827.67	
11	BTC	2019-03-09	3828.37	3859.8	3918	3778.52	
12	BTC	2019-03-10	3898.87	3841.89	3948.88	3832.14	
13	BTC	2019-03-11	3899.66	3916.76	3921.93	3865.92	
14	BTC	2019-03-12	3851.25	3899.46	3913.46	3819.93	

8.3.3 다중 채널 비트코인 가격 예측

- [프로그램 8-3] 다중 채널 비트코인 가격 예측
 - 윈도우에 맞게 시계열 데이터 준비 ($w = 7, h = 1$)

```
1 # 학습데이터와 테스트데이터가 나눠 제공되기 때문에, 교재와 차이가 있음
2 def seq2dataset(seq, window, horizon, isTrain=True):
3     """
4     seq: np.array format
5     window: sampling step
6     """
7     X=[]; Y=[]
8
9     for i in range(len(seq)-(window+horizon)+1):
10         x=seq[i:(i+window)]
11         X.append(x);
12
13         if isTrain:
14             y=(seq[i+window+horizon-1])
15             Y.append(y)
16
17     if isTrain:
18         return np.array(X), np.array(Y)
19     else:
20         return np.array(X)
```

+ Code + Markdown

```
[7]:
1 w = 7
2 h = 1
3
4 np_train = pd_train.to_numpy()
5 np_test = pd_test.to_numpy()
6
7 X_train, Y_train = seq2dataset(np_train, w, h)
8 X_test = seq2dataset(np_test, w, h, isTrain=False)
9
10 print(X_train.shape, Y_train.shape)
11 print(X_test.shape)
12
```

(293, 7, 4) (293, 4)
(58, 7, 4)

	closing price	open price	high price	low price
0	3816.6	3814.6	3883.7	3783.3
1	3821.9	3816.7	3855.8	3816.4
2	3823.1	3821.9	3843.2	3783.6
3	3809.5	3823.2	3836.6	3789.7
4	3715.9	3809.7	3828.4	3681.8
5	3857.2	3715.9	3873.2	3705.7
6	3863.0	3857.2	3887.3	3816.7
7	3875.1	3863.1	3907.4	3847.9
8	3865.9	3875.1	3929.0	3810.7
9	3944.3	3865.6	3964.0	3859.7

Train_X

```
array([[3816.6, 3814.6, 3883.7, 3783.3],
       [3821.9, 3816.7, 3855.8, 3816.4],
       [3823.1, 3821.9, 3843.2, 3783.6],
       ...,
       [3715.9, 3809.7, 3828.4, 3681.8],
       [3857.2, 3715.9, 3873.2, 3705.7],
       [3863. , 3857.2, 3887.3, 3816.7]],
      dtype=float64)
```

Train_Y

```
[32... array([[3875.1, 3863.1, 3907.4, 3847.9],
          [3865.9, 3875.1, 3929. , 3810.7],
          [3944.3, 3865.6, 3964. , 3859.7],
          ...,
          [7495.8, 7156.3, 7501.1, 7142. ],
          [7322.8, 7496.2, 7684. , 7276.2],
          [7268.3, 7322.4, 7433.2, 7185.7]])
```

8.5 자연어 처리

- 자연어 처리(NLP: natural language processing)란?
 - 인간이 구사하는 언어를 자동으로 처리하는 프로그램을 만드는 연구 분야임
- 자연어 처리 분야 응용
 - 한국어를 영어로 번역하는 기술
 - 영화평에 달린 댓글을 분석해 흥행을 추정하는 기술
 - 고객의 음성에서 의도를 이해하여 적절히 응대하는 챗봇 기술
 - 소설이나 시를 쓰는 창작 인공지능 기술

딕셔너리를 이용하여 텍스트를 숫자 코드로 변환

- Freshman loves python. → [2 3 1]
- We teach python to freshman. → [4 5 1 6 2]
- How popular is python? → [7 8 9 1]

8.5.1 텍스트 데이터에 대한 이해

- 자연어는 텍스트로 표현되며, 아래의 문장은 영화평 데이터셋인 IMDB의 예제 문장임

Once again Mr. Costner has dragged out a movie for far longer than necessary. Aside from the terrific sea rescue sequences, of which there are very few I just did not care about any of the characters. Most of us have ghosts in the closet, and Costner's character are realized early on, and then forgotten until much later, by which time I did not care,

- 텍스트 데이터의 특성
 - 시계열 데이터로서 시간 정보가 있고 샘플마다 길이가 다르다는 기본 성질
 - 그 외 특성
 - 심한 잡음
 - 숫자, 특수 기호, 외국어, 비속어(ㅋㅋㅋ), 비문, 사투리, 오타, 띄어쓰기 오류 등
 - 형태소 분석 필요
 - 형태소란 더 나누면 의미가 훼손되는 문장의 최소 단위로, **한글은 다른 언어에 비해 형태소 분석이 어려움.**
 - 예) “너는 누구에게 한 번이라도” → “너+는 누구+에게 한 번+이라도”

8.5.1 텍스트 데이터에 대한 이해

- 텍스트 데이터의 특성
 - 그 외 특성
 - 구문론과 의미론
 - 구문론 : 형태소 분석처럼 문법적인 구조를 따지는 일
 - 의미론: 각 단어의 의미, 단어와 단어가 결합해 만들어 내는 의미를 이해하는 일
 - 의미론까지 분석해야 똑똑한 챗봇을 만들 수 있음
 - 다양한 언어 특성
 - 성별유무 : 영어에는 명사에 성별이 없지만, 독일어에는 명사에 성별이 있음
 - 신경망에 입력하려면 기호를 수치로 변환 해야함
 - 신경망은 수치 값을 입력으로 받기 때문에 원핫 코드나 적당한 코드로 변환 필요
 - 현대 딥러닝은 단어 임베딩 기술을 사용해 벡터 공간으로 변환

8.5.1 텍스트 데이터에 대한 이해

- 원한 코드 표현으로 변환하는 절차

- 말뭉치(Corpus) 사례

```
Freshman loves python.  
We teach python to freshman.  
How popular is Python?
```

- 단어(word) 수집(괄호 속은 빈도 수)

- python(3), freshman(2), loves(1), we(1), teach(1), to(1), how(1), popular(1), is(1)

- 파이썬의 자료구조인 딕셔너리를 이용한 표현(빈도수에 따른 순위 부여)

- {'python':1, 'freshman':2, 'loves':3, 'we':4, 'teach':5, 'to':6, 'how':7, 'popular':8, 'is':9}

- 딕셔너리를 이용하여 텍스트를 숫자 코드로 변환

- Freshman loves python. → [2 3 1]
 - We teach python to freshman. → [4 5 1 6 2]
 - How popular is python? → [7 8 9 1]

8.5.1 텍스트 데이터에 대한 이해

- 원핫 코드 표현

- 자연어 처리 기술에서는 시계열 데이터 [2 3 1] 이나 [4 5 1 6 2]를 그대로 신경망에 입력하지 않고 원핫 코드 또는 보다 발전된 표현 방법인 단어 임베딩으로 변환한 후 신경망에 입력함

- 원핫 코드 예

```
[231]    → [[010000000] [001000000] [100000000]]
[45162]  → [000100000] [000010000] [100000000] [000001000] [010000000]
[7891]   → [000000100] [000000010] [000000001] [100000000]
```

- 원핫 코드의 문제점

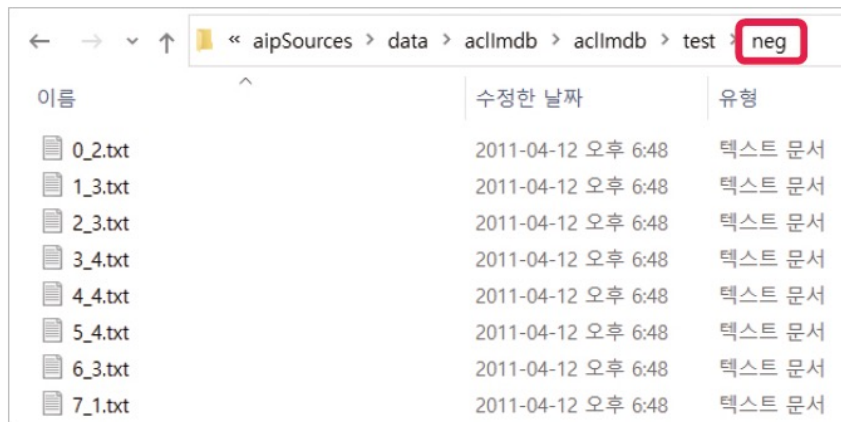
- 사전 크기가 크면 원핫 코드는 희소 벡터가 되어 메모리 낭비가 심해짐
- 단어 사이의 연결 관계를 반영하지 못함
 - 예) king과 queen, bridegroom과 bride의 관계 표현 불가능
- 현대적인 표현 기법 단어 임베딩은 나중에 다룸 (Word2Vec, Glove 등)

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

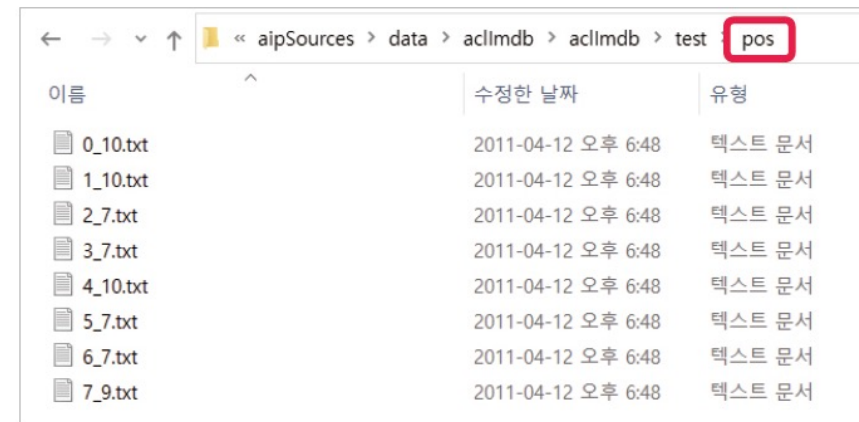
- 텍스트 데이터
 - 영화를 평가한 댓글을 모아둔 IMDB 데이터셋
 - 50000개의 댓글을 긍정 평가와 부정 평가로 레이블링
 - 감정 분류(sentiment classification) 문제에 주로 사용
 - 다양한 토픽의 뉴스를 모아둔 Reuters 데이터셋
 - 로이터 통신의 뉴스 11228개를 46개 토픽으로 레이블링
 - 토픽 분류 문제에 주로 사용
- 지금부터 IMDB 데이터셋으로 실습 진행

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- IMDB 원본 데이터 살펴보기
 - <http://mng.bz/0tlo>에 접속하여 원본 데이터 다운로드
 - 소스 프로그램이 있는 폴더에 data라는 폴더 만들어 거기에 저장



이름	수정한 날짜	유형
0_2.txt	2011-04-12 오후 6:48	텍스트 문서
1_3.txt	2011-04-12 오후 6:48	텍스트 문서
2_3.txt	2011-04-12 오후 6:48	텍스트 문서
3_4.txt	2011-04-12 오후 6:48	텍스트 문서
4_4.txt	2011-04-12 오후 6:48	텍스트 문서
5_4.txt	2011-04-12 오후 6:48	텍스트 문서
6_3.txt	2011-04-12 오후 6:48	텍스트 문서
7_1.txt	2011-04-12 오후 6:48	텍스트 문서



이름	수정한 날짜	유형
0_10.txt	2011-04-12 오후 6:48	텍스트 문서
1_10.txt	2011-04-12 오후 6:48	텍스트 문서
2_7.txt	2011-04-12 오후 6:48	텍스트 문서
3_7.txt	2011-04-12 오후 6:48	텍스트 문서
4_10.txt	2011-04-12 오후 6:48	텍스트 문서
5_7.txt	2011-04-12 오후 6:48	텍스트 문서
6_7.txt	2011-04-12 오후 6:48	텍스트 문서
7_9.txt	2011-04-12 오후 6:48	텍스트 문서

그림 8-16 IMDB 데이터셋의 폴더 구조 - neg와 pos 폴더

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 IMDB 데이터셋을 통해 원본 텍스트 데이터 읽기

```
[4]: df = pd.read_csv('/kaggle/input/mydatafiles123/IMDB Dataset.csv')
df
```

```
[4]:
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive
...	...	...
49995	I thought this movie did a down right good job...	positive
49996	Bad plot, bad dialogue, bad acting, idiotic di...	negative
49997	I am a Catholic taught in parochial elementary...	negative
49998	I'm going to have to disagree with the previou...	negative
49999	No one expects the Star Trek movies to be high...	negative

50000 rows x 2 columns

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 one-hot 변환하기**
 - 입력데이터 예시

```
>>> corpus = [  
...     'This is the first document.',  
...     'This document is the second document.',  
...     'And this is the third one.',  
...     'Is this the first document?',  
... ]
```

- 추출된 단어 사전 (vocabulary)

```
array(['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third',  
      'this'], ...)
```

- 단어 사전으로 만든 One-hot 벡터

```
[[0 1 1 1 0 0 1 0 1]  
 [0 1 0 1 0 1 1 0 1]  
 [1 0 0 1 1 0 1 1 1]  
 [0 1 1 1 0 0 1 0 1]]
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 one-hot 변환하기**

[29]:

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    'This is the first document.',
    'This document is the second document.',
    'And this is the third one.',
    'Is this the first document?',
]

vectorizer = CountVectorizer(binary=True)
X = vectorizer.fit_transform(corpus)
vectorizer.get_feature_names_out()
print(X.toarray())
```

```
[[0 1 1 1 0 0 1 0 1]
 [0 1 0 1 0 1 1 0 1]
 [1 0 0 1 1 0 1 1 1]
 [0 1 1 1 0 0 1 0 1]]
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 BoW로 표현하기**
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론
 - TF(Term Frequency) 방법론
 - 단어의 등장 횟수에 비례하여 가중치 부여하는 방법론

```
from sklearn.feature_extraction.text import CountVectorizer

one_hot_vectorizer = CountVectorizer() |
one_hot = one_hot_vectorizer.fit_transform(x_train).toarray() # 원-핫 벡터
one_hot_vectorizer.get_feature_names_out() # 학습된 단어 모음
one_hot_vectorizer.vocabulary_ # 단어당 출현 빈도
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 BoW로 표현하기**
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론
 - TF-IDF(Term Frequency-Inverse Document Frequency) 방법론
 - 빈도가 높지만 문서의 특징 정보가 잘 담기지 않는 단어
 - 빈도가 낮지만 문서의 특징 정보를 잘 담고 있는 단어
 - 이런 상황을 고려한 역문서 빈도

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()
tfidf = tfidf_vectorizer.fit_transform(x_train).toarray() # TF-IDF 벡터
vocab = tfidf_vectorizer.get_feature_names_out() # 학습된 단어 모음
tfidf_vectorizer.vocabulary_ # 단어당 출현 빈도
```


8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 BoW로 표현하기**
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론

- 방법론

```
{'story': 63422,  
'of': 46680,  
'man': 40591,  
'who': 72904,  
'has': 29999,  
'unnatural': 69909,  
'feelings': 24126,  
'for': 25450,  
'pig': 49917,  
'starts': 62913,  
'out': 47449,  
'with': 73342,  
'opening': 47033,  
'scene': 57873,  
'that': 66322,  
'is': 34585,  
'terrific': 66156,  
'example': 22881,  
'absurd': 1343,  
'comedy': 13498,  
'formal': 25588,  
'orchestra': 47178,  
'audience': 4956,  
'turned': 68619,  
'into': 34255,  
'an': 3167,  
'insane': 33779,  
'violent': 71467,  
'mob': 43335,
```

```
from sklearn.feature_extraction.text import CountVectorizer  
  
cv = CountVectorizer() |  
vectorizer.fit_transform(x_train).toarray() # 원-핫 벡터  
get_feature_names_out() # 학습된 단어 모음  
vocabulary_ # 단어당 출현 빈도
```

Frequency-Inverse Document Frequency) 방법론

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
tfidf = TfidfVectorizer()  
rizer.fit_transform(x_train).toarray() # TF-IDF 벡터  
rizer.get_feature_names_out() # 학습된 단어 모음  
cabulary_ # 단어당 출현 빈도
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 BoW로 표현하기**
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론 : (1) 데이터 로드

```
[2]: # imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv
df = pd.read_csv('/kaggle/input/d/yukyungchoi/imdb-dataset/IMDB Dataset.csv')
df
```

```
[2]:
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive
...	...	...
49995	I thought this movie did a down right good job...	positive
49996	Bad plot, bad dialogue, bad acting, idiotic di...	negative
49997	I am a Catholic taught in parochial elementary...	negative
49998	I'm going to have to disagree with the previou...	negative
49999	No one expects the Star Trek movies to be high...	negative

50000 rows x 2 columns

<https://www.kaggle.com/code/yukyungchoi/2024-idl-imdb-sklearn>

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 BoW로 표현하기**
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론 : (2) 데이터 가공

```
[3]: df['review'] = df['review'].astype(str)
      reviews = df['review']
      reviews
```

```
[3]: 0      One of the other reviewers has mentioned that ...
      1      A wonderful little production. <br /><br />The...
      2      I thought this was a wonderful way to spend ti...
      3      Basically there's a family where a little boy ...
      4      Petter Mattei's "Love in the Time of Money" is...
      ...
      49995 I thought this movie did a down right good job...
      49996 Bad plot, bad dialogue, bad acting, idiotic di...
      49997 I am a Catholic taught in parochial elementary...
      49998 I'm going to have to disagree with the previou...
      49999 No one expects the Star Trek movies to be high...
      Name: review, Length: 50000, dtype: object
```

```
[4]: sentiment = df['sentiment']
      sentiment[df['sentiment']=='positive'] = True
      sentiment[df['sentiment']=='negative'] = False
      sentiment = sentiment.astype(str)
      sentiment
```

```
[4]: 0      True
      1      True
      2      True
      3      False
      4      True
      ...
      49995 True
      49996 False
      49997 False
      49998 False
      49999 False
      Name: sentiment, Length: 50000, dtype: object
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 BoW로 표현하기**
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론 : (3) 특징 추출

```
[ ]: # 리뷰 전처리 추가 가능 - 하지만 전처리 없이 베이스 잡아보겠음
```

▷

```
# 텍스트 데이터 BOW로 변환
from sklearn.feature_extraction.text import CountVectorizer

#count_vec = CountVectorizer(binary=False, ngram_range=(2, 3), max_features=20000)
count_vec = CountVectorizer(binary=False, max_features=20000)

count_vec_x = count_vec.fit_transform(reviews).toarray()
count_vec_x = count_vec_x.astype(np.int8)
count_vec_x.shape
```

```
[7]: (50000, 20000)
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- 제공된 데이터셋을 통해 **원본 데이터 BoW로 표현하기**
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론 : (4) 모델 학습 및 평가

```
[8]: from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(count_vec_x, sentiment, test_size = 0.20, random_state = 7)
```

```
[9]: from sklearn.linear_model import SGDClassifier

sgdclssifier_model = SGDClassifier(loss='hinge',penalty='l2', alpha=0.001,
                                   l1_ratio=0.15, fit_intercept=True, max_iter=20000,
                                   tol=0.001, shuffle=True, verbose=0, epsilon=0.001,
                                   n_jobs=None, random_state=7, learning_rate='optimal',
                                   eta0=0.0, power_t=0.5, early_stopping=False,
                                   validation_fraction=0.1, n_iter_no_change=5,
                                   class_weight=None, warm_start=False, average=False)

sgdclssifier_model.fit(x_train,y_train)
```

```
[9]: SGDClassifier
SGDClassifier(alpha=0.001, epsilon=0.001, max_iter=20000, random_state=7)
```

+ Code + Markdown

```
[10]: print(f'Train score:{format(sgdclssifier_model.score(x_train,y_train))}',end = '\n\n')
      print(f'Test score:{format(sgdclssifier_model.score(x_test,y_test))}',end = '\n\n')
```

Train score:0.95795

Test score:0.8916

<https://www.kaggle.com/code/yukyungchoi/2024-idl-imdb-sklearn>

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- IMDB 원본 텍스트 데이터 토큰화 : 전처리

- 토큰화(tokenization) : 주어진 코퍼스(corpus)에서 토큰(token)을 불리는 단위로 나누는 작업을 말함. 토큰은 단위가 상황에 따라 다르지만, 보통 의미 있는 단위로 토큰을 정의함

- 토큰화의 예

- 입력: Time is an illusion. Lunchtime double so!
- 출력 : "Time", "is", "an", "illusion", "Lunchtime", "double", "so"

- ✓ 구두점(punctuation)과 같은 문자는 제외시키는 간단한 단어 토큰화 작업
- ✓ 구두점이란 마침표(.), 쉼표(,), 물음표(?), 세미콜론(;), 느낌표(!) 등과 같은 기호

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- IMDB 원본 텍스트 데이터 토큰화 : 전처리

[16]:

```
# 리뷰 전처리 추가
import re
import string

def remove_tags(string):
    result = re.sub('<.*?>', '', string)
    return result

reviews = reviews.apply(lambda i: remove_tags(i))
```

[15]:

reviews

```
[15... 0      One of the other reviewers has mentioned that ...
1      A wonderful little production. <br /><br />The...
2      I thought this was a wonderful way to spend ti...
3      Basically there's a family where a little boy ...
4      Petter Mattei's "Love in the Time of Money" is...

      ...
49995  I thought this movie did a down right good job...
49996  Bad plot, bad dialogue, bad acting, idiotic di...
49997  I am a Catholic taught in parochial elementary...
49998  I'm going to have to disagree with the previou...
49999  No one expects the Star Trek movies to be high...
Name: review, Length: 50000, dtype: object
```

[17]:

reviews

```
[17... 0      One of the other reviewers has mentioned that ...
1      A wonderful little production. The filming tec...
2      I thought this was a wonderful way to spend ti...
3      Basically there's a family where a little boy ...
4      Petter Mattei's "Love in the Time of Money" is...

      ...
49995  I thought this movie did a down right good job...
49996  Bad plot, bad dialogue, bad acting, idiotic di...
49997  I am a Catholic taught in parochial elementary...
49998  I'm going to have to disagree with the previou...
49999  No one expects the Star Trek movies to be high...
Name: review, Length: 50000, dtype: object
```

<https://www.kaggle.com/code/yukyungchoi/2024-idl-imdb-sklearn-nltk>

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- IMDB 원본 텍스트 데이터 토큰화 : 전처리



```
reviews = reviews.str.replace(r'^\w\d\s', ' ', regex=True)
reviews = reviews.str.replace(r'\s+', ' ', regex=True)
reviews = reviews.str.replace(r'^\s+|\s+?$', '', regex=True)
```

[17]:

reviews

```
[17... 0      One of the other reviewers has mentioned that ...
      1      A wonderful little production. The filming tec...
      2      I thought this was a wonderful way to spend ti...
      3      Basically there's a family where a little boy ...
      4      Petter Mattei's "Love in the Time of Money" is...

      ...
49995      I thought this movie did a down right good job...
49996      Bad plot, bad dialogue, bad acting, idiotic di...
49997      I am a Catholic taught in parochial elementary...
49998      I'm going to have to disagree with the previou...
49999      No one expects the Star Trek movies to be high...
Name: review, Length: 50000, dtype: object
```

[19]:

reviews

```
[19... 0      One of the other reviewers has mentioned that ...
      1      A wonderful little production The filming tech...
      2      I thought this was a wonderful way to spend ti...
      3      Basically there s a family where a little boy ...
      4      Petter Mattei s Love in the Time of Money is a...

      ...
49995      I thought this movie did a down right good job...
49996      Bad plot bad dialogue bad acting idiotic direc...
49997      I am a Catholic taught in parochial elementary...
49998      I m going to have to disagree with the previou...
49999      No one expects the Star Trek movies to be high...
Name: review, Length: 50000, dtype: object
```


8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- IMDB 원본 텍스트 데이터 토큰화 : 전처리

```
[ ]: reviews = reviews.str.lower()
```

```
[19]: reviews
[19... 0      One of the other reviewers has mentioned that ...
      1      A wonderful little production The filming tech...
      2      I thought this was a wonderful way to spend ti...
      3      Basically there s a family where a little boy ...
      4      Petter Mattei s Love in the Time of Money is a...
      ...
      49995 I thought this movie did a down right good job...
      49996 Bad plot bad dialogue bad acting idiotic direc...
      49997 I am a Catholic taught in parochial elementary...
      49998 I m going to have to disagree with the previou...
      49999 No one expects the Star Trek movies to be high...
      Name: review, Length: 50000, dtype: object
```

▷

```
reviews
[21... 0      one of the other reviewers has mentioned that ...
      1      a wonderful little production the filming tech...
      2      i thought this was a wonderful way to spend ti...
      3      basically there s a family where a little boy ...
      4      petter mattei s love in the time of money is a...
      ...
      49995 i thought this movie did a down right good job...
      49996 bad plot bad dialogue bad acting idiotic direc...
      49997 i am a catholic taught in parochial elementary...
      49998 i m going to have to disagree with the previou...
      49999 no one expects the star trek movies to be high...
      Name: review, Length: 50000, dtype: object
```

8.5.2 텍스트 데이터 사례: 영화평 데이터셋 IMDB

- IMDB 원본 텍스트 데이터 토큰화 : 전처리
 - Sklearn 라이브러리를 이용한 Bag of Words 방법론 + 전처리 결과 : 약간 성능 개선

```
[23]: from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(count_vec_x, sentiment, test_size = 0.20, ran
```

```
[24]: from sklearn.linear_model import SGDClassifier

sgdclssifier_model = SGDClassifier(loss='hinge', penalty='l2', alpha=0.001,
                                   l1_ratio=0.15, fit_intercept=True, max_iter=20000,
                                   tol=0.001, shuffle=True, verbose=0, epsilon=0.001,
                                   n_jobs=None, random_state=7, learning_rate='optimal',
                                   eta0=0.0, power_t=0.5, early_stopping=False,
                                   validation_fraction=0.1, n_iter_no_change=5,
                                   class_weight=None, warm_start=False, average=False)

sgdclssifier_model.fit(x_train, y_train)
```

```
[24...] SGDClassifier
SGDClassifier(alpha=0.001, epsilon=0.001, max_iter=20000, random_state=7)
```

```
[25]: print(f'Train score:{format(sgdclssifier_model.score(x_train,y_train))}',end = '\n\n')
      print(f'Test score:{format(sgdclssifier_model.score(x_test,y_test))}',end = '\n\n')
```

Train score:0.957525

Test score:0.8922

<https://www.kaggle.com/code/yukyungchoi/2024-idl-imdb-sklearn-nltk>

8.5.3 단어 임베딩

- 텍스트 데이터를 신경망에 입력하고 표현하는 방법
 - 원핫 코드
 - 사전이 매우 커서 아주 큰 희소 텐서가 입력됨
 - 단어의 의미를 전혀 반영하지 못함
 - 단어 임베딩 (원핫코드 단점 해결)
 - 단어를 저차원 공간의 벡터로 표현하는 기법
 - 보통 수백 차원을 사용 (즉, 아주 큰 텐서가 아님)
 - 밀집 벡터임 (즉, 희소 텐서가 아님)
 - 단어의 의미를 표현
 - 신경망 학습을 통해 알아냄

[원핫 코드]

사전 크기([프로그램 8-7(a)]의 dic_siz)가 10000인 경우

"python" → $\overbrace{(1\ 0\ 0\ 0\ 0\ \dots\ 0\ 0\ 0\ \dots\ 0\ 0\ 0)}^{10000\text{개 요소}}$

[단어 임베딩]

단어 임베딩 공간의 차원([프로그램 8-7(b)]의 embed_space_dim)이 16인 경우

"python" → $\overbrace{(0.01\ 0.23\ 0.0\ \dots\ 0.002)}^{16\text{개 요소}}$

8.5.3 단어 임베딩

- 텍스트 데이터를 신경망에 입력하고 표현하는 방법
 - 원핫 코드
 - 사전이 매우 커서 아주 큰 희소^{sparse} 텐서가 입력됨
 - 단어의 의미를 전혀 반영하지 못함
 - 단어 임베딩 (원핫코드 단점 해결)
 - 단어를 저차원 공간의 벡터로 표현하는 기법
 - 보통 수백 차원을 사용 (즉, 아주 큰 텐서가 아님)
 - 밀집 벡터임 (즉, 희소 텐서가 아닌 분산^{distributed} 텐서)
 - 단어의 의미를 표현 (단어의 의미를 반영한 공간을 신경망 학습으로 생성)
 - 신경망 학습을 통해 알아냄

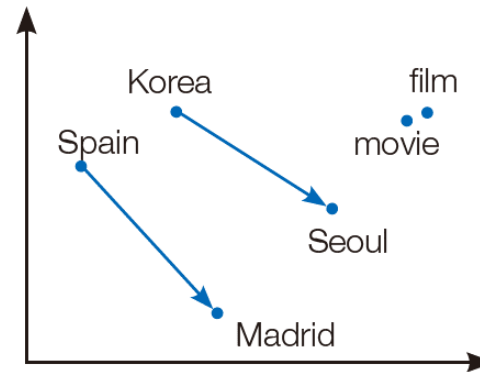


그림 8-17 단어 임베딩의 원리

8.5.3 MLP로 분류 : sklearn

- [프로그램 8-7(b)] TF 카운트 사용하여 IMDB 데이터를 MLP로 학습
 - 샘플사이즈 512
 - MLP hidden = 32

```
# 텍스트 데이터 BOW로 변환
from sklearn.feature_extraction.text import CountVectorizer

#count_vec = CountVectorizer(binary=False, ngram_range=(2, 3), max_features=20000)
count_vec = CountVectorizer(binary=False, max_features=512)

count_vec_x = count_vec.fit_transform(reviews).toarray()
count_vec_x = count_vec_x.astype(np.int8)
count_vec_x.shape

# CPU MLP는 느림
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# clf = MLPClassifier()
clf = MLPClassifier(solver='adam', hidden_layer_sizes=(32), max_iter=10).fit(x_train, y_train)
y_pred = clf.predict(x_test)
```

<https://www.kaggle.com/code/yukyungchoi/2024-idl-imdb-sklearn-mlp>

8.5.3 MLP로 분류 : Pytorch

- [프로그램 8-7(b)] TF 카운트 사용하여 IMDB 데이터를 **MLP로 학습**
 - 샘플사이즈 512
 - MLP hidden = 32

```
1 class Net(torch.nn.Module):
2     def __init__(self, in_dim, hid_dim, out_dim):
3         super(Net, self).__init__()
4         self.l1 = nn.Linear(in_dim, hid_dim)
5         self.l2 = nn.Linear(hid_dim, out_dim)
6         self.relu = nn.ReLU()
7         self.layers = nn.Sequential(self.l1, self.relu, self.l2)
8     def forward(self, x):
9         out = self.layers(x)
10        return out
```

```
12
13 # model (MLP)
14 input_dim=512
15 hidden_dim=32
16 output_dim=2
17
18 model = Net(input_dim, hidden_dim, output_dim)
19 model = model.to(device)
20 loss = torch.nn.CrossEntropyLoss()
21 loss = loss.to(device)
22
```

<https://www.kaggle.com/code/yukyungchoi/2024-idl-imdb-pytorch-mlp>

8.5.3 MLP로 분류 : sklearn

- [프로그램 8-8] 캐글 리더보드 이용하여 IMDB 긍정/부정 리뷰 문제 해결하기

- 앞의 예제를 참고하여 해결할 것

- 베이스라인 기본 세팅 동일

```
clf = MLPClassifier(solver='adam', hidden_layer_sizes=(32), max_iter=10).fit(X_train, y_train)
```

(시간 관계상 짧게 학습하였음을 참고)

- 주의할 점

- 답안 제출 시, Positive/Negative 로 제출해야 함

	A	B
1	id	sentiment
2	0	negative
3	1	negative
4	2	negative
5	3	positive
6	4	negative
7	5	negative
8	6	negative
9	7	negative
10	8	negative
11	9	negative
12	10	negative
13	11	negative

<https://www.kaggle.com/t/34bd873dfd824d9091389ae834ae0f9a>

8.5.5 word2vec 과 Glove

- 단어를 벡터 공간에 표현하는 단어 임베딩 기술

- 오래 전부터 연구되어온 아이디어
- 영국의 언어학자 퍼스는 “You shall know a word by the company it keeps.” 라는 말을 통해 단어 간의 상호 작용이 매우 중요하다는 통찰을 남김
 - 예) 영화라는 단어는 아카데미, 흥행 등의 단어와 함께 등장할 가능성 높음
- 고전적인 기법들

카운트기반 임베딩 ▪ 1960년대 : TF (term frequency) 단어가 출현하는 빈도를 세어 벡터로 표현

카운트기반 임베딩 ▪ 1980년대 : LSA (latent semantic analysis) 행은 단어이고 열은 문서 번호인 행렬에 단어 빈도를 채운 다음 SVD 기법을 이용해 정보 손실을 최소로 유지한 채 차원 축소를 달성하는 기법

- 2000년대 : 신경망 기법들

- 2010년대 : 딥러닝 기법들

예측기반 임베딩 ▪ 구글에서 공개한 word2vec

- 1000억 개 가량의 뉴스를 모아둔 데이터셋으로 학습, 300만 개 가량의 단어를 각각 300차원 공간에 표현

학습기반 임베딩 ▪ 스탠퍼드에서 공개한 Glove

- 위키피디아 문서 데이터를 사용해 학습, 40만 개 가량의 단어를 50차원, 100차원, 200차원, 300차원 공간에 표현

8.5.5 word2vec 과 Glove

- 단어를 벡터 공간에 표현하는 단어 임베딩 기술
 - LSA와 같은 카운트 기반 임베딩 기법은 코퍼스의 전체적인 통계 정보를 고려하기 때문에 [왕:남자=여왕:? (정답은 여자)] 와 같은 단어 의미의 유추 작업에는 성능이 떨어짐
 - Word2Vec 와 같은 예측 기반 임베딩 기법은 단어 간 유추 작업은 뛰어나지만, 임베딩 벡터가 윈도우 크기 내에서만 주변 단어를 고려하기 때문에 코퍼스의 전체적인 통계 정보를 반영하지 못함
 - Glove와 같은 학습 기반 임베딩 기법은 앞서 소개한 두 임베딩 기법의 한계를 극복하기 위해 제안되었음
- 학습 기반 임베딩 기법
 - 사전 훈련된 단어 임베딩 (Pretrained word embedding)
 - 구글 뉴스, 위키피디아, 커먼 크롤 과 같은 대규모 말뭉치로 학습된 모델
 - 대표적인 방법론은 스탠퍼드에서 공개한 Glove (Global Vectors for Word Representation)
 - 핵심 아이디어 : 임베딩 된 중심 단어와 주변 단어 벡터의 내적이 전체 코퍼스에서의 동시 등장 확률이 되도록 만드는 것

8.5.5 word2vec 과 Glove

- Glove 학습을 위한 문장 토큰화 예시

▷

```
# 토큰화를 위한 패키지 다운로드
import nltk
nltk.download('punkt')

# 하나의 문장 토큰화 예시
from nltk import word_tokenize

sentence = "The Matrix is everywhere its all around us, here even in this room."
words = word_tokenize(sentence)
print(type(words), len(words))
print(words)
|

#여러 개의 문장으로 된 입력 데이터를 문장별로 단어 토큰화
from nltk import word_tokenize, sent_tokenize

def tokenize_text(text):

    # 문장별로 분리 토큰
    sentences = sent_tokenize(text)
    # 분리된 문장별 단어 토큰화
    word_tokens = [word_tokenize(sentence) for sentence in sentences]
    return word_tokens

text_sample = 'The Matrix is everywhere its all around us, here even in this room. \
You can see it out your window or on your television. \
You feel it when you go to work, or go to church or pay your taxes.'

word_tokens = tokenize_text(text_sample)
print(type(word_tokens), len(word_tokens))
print(word_tokens)
```

```
<class 'list'> 15
['The', 'Matrix', 'is', 'everywhere', 'its', 'all', 'around', 'us', ',', 'here', 'even', 'in', 'this', 'room', '.']
```

8.5.5 word2vec 과 Glove

▪ Glove 학습 예시

```
▶ # 글로브 패키지 사용을 위한 설치
!pip install glove-python3

Collecting glove-python3
  Downloading glove-python3-0.6.3.tar.gz (1.1 MB)
    Installing build dependencies: finished with status 'done'
    Getting requirements to build wheel: finished with status 'done'
    Preparing metadata (pyproject.toml): finished with status 'done'
  Installing the package: finished with status 'done'

[ ]: # 토큰화 전
x_train

# 토큰화를 위한 전처리
parse_text = '\n'.join(x_train)
# 토큰화
word_tokens = tokenize_text(parse_text)
#print(type(word_tokens), len(word_tokens))
#print(word_tokens)

[87]: # Glove 임베딩
from glove import Corpus, Glove

corpus = Corpus()

# 훈련 데이터로부터 GloVe에서 사용할 동시 등장 행렬 생성
corpus.fit(word_tokens, window=5)
glove = Glove(no_components=100, learning_rate=0.05)

# 학습에 이용할 스레드의 개수는 4로 설정, 에포크는 20.
glove.fit(corpus.matrix, epochs=20, no_threads=4, verbose=True)
glove.add_dictionary(corpus.dictionary)

≡ print(glove.most_similar("man"))

[('woman', 0.9376907431819917), ('girl', 0.8996554764817649), ('boy', 0.8711861755954549), ('name
d', 0.8500337277891531)]
```

settings

임베딩벡터 : 100차원

+ Code + Markdown

8.5.5 word2vec 과 Glove

- Torchtext를 이용한 임베딩

```
:  
from torchtext.vocab import GloVe  
import torch.nn  
glove = GloVe()  
my_embeddings = torch.nn.Embedding.from_pretrained(glove.vectors, freeze=True)
```

```
.vector_cache/glove.840B.300d.zip: 2.18GB [07:04, 5.13MB/s]  
100%|██████████| 2196016/2196017 [05:13<00:00, 7007.04it/s]
```

```
test_text1 = ["Hello", "world"]  
test_text2 = ["Hello", "my", "friend"]  
  
embedded_text1 = glove.get_vecs_by_tokens(test_text1)  
embedded_text2 = glove.get_vecs_by_tokens(test_text2)
```

+ Code

+ Markdown

```
:  
print(embedded_text1.shape)  
print(embedded_text2.shape)
```

```
torch.Size([2, 300])  
torch.Size([3, 300])
```